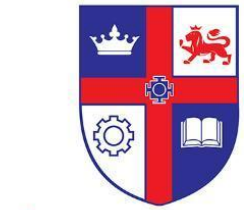
LONDON METROPOLITAN UNIVERSITY



Islington college
(इस्लिङ्टन कलेज)

CS4001NI Programming

60% Individual Coursework

2025 Spring

Student Name: Dilasha Vaidya

London Met ID: 24046749

College ID: NP01CP4A240203

Group: C8

Assignment Due Date: Friday, May 16, 2025

Assignment Submission Date: Friday, May 16, 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Similarity report – Turnitin

Document 1.docx

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:84998787

Submission Date
Mar 8, 2025, 3:51 PM GMT+5:45

Download Date
Mar 8, 2025, 3:52 PM GMT+5:45

File Name
Document 1.docx

File Size
3.5 KB

4 Pages

483 Words

3,006 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 1 Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
2% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

Table of Contents

Table of Figures

Table of tables

Introduction

Introduction to the project

Implementing a real-world gym membership system with Java's object-oriented ideas is the main goal of this project. In order to differentiate between various membership kinds, the system is built with a basic class called ‘GymMember’ and two subclasses called ‘RegularMember’ and ‘PremiumMember’. For effective member data management and storage, the project uses an Array List. The integration of a Graphical User Interface (GUI) further enhances the user-friendliness of the process of adding, viewing, and managing gym members. Executed via the command prompt, the program will demonstrate how object-oriented programming (OOP) concepts like inheritance, encapsulation, and polymorphism may be used to address real-world issues. By creating this system, we may gain a greater understanding of how fitness facilities leverage digital solutions to automate data handling, enhance user interaction, and streamline membership management.

Goals and Objectives

- **Apply the Principles of Object-Oriented Programming (OOP):**
  Create an organized Gym Membership system by utilizing polymorphism, encapsulation, and inheritance.
- **Create a User-Friendly GUI:**
  Provide an interactive interface that makes it easy to add, view, and manage gym members.
- **Effectively save and Manage Data:**
  To save and retrieve gym member information dynamically, use an Array List.
- **Develop Real-World Application Skills:**
  Utilize Java programming techniques to create a workable membership management system for fitness establishments.

Verdict

This project effectively uses the principles of Object-Oriented Programming (OOP) to create a Java gym membership system. For effective data administration, it combines an Array List with a Graphical User Interface (GUI). In addition to improving user interaction, the system offers a practical method for managing gym memberships.

Discussion and Analysis

BlueJ

Specifically created for learning and creating Java applications, BlueJ is an integrated development environment (IDE). BlueJ, which offers a straightforward yet effective interface for implementing Object-Oriented Programming (OOP) concepts, was utilized in this project to write, compile, and test the Gym Membership System. Because the software made it simple to visualize class structures, managing the relationships between the "GymMember," "RegularMember," and "PremiumMember" classes was made easier. Debugging and improving the system also benefited from BlueJ's interactive object generation and method testing features. Its integrated compiler and visual depiction of class hierarchies aided in comprehending and refining the project structure, resulting in a more streamlined and user-friendly development process.

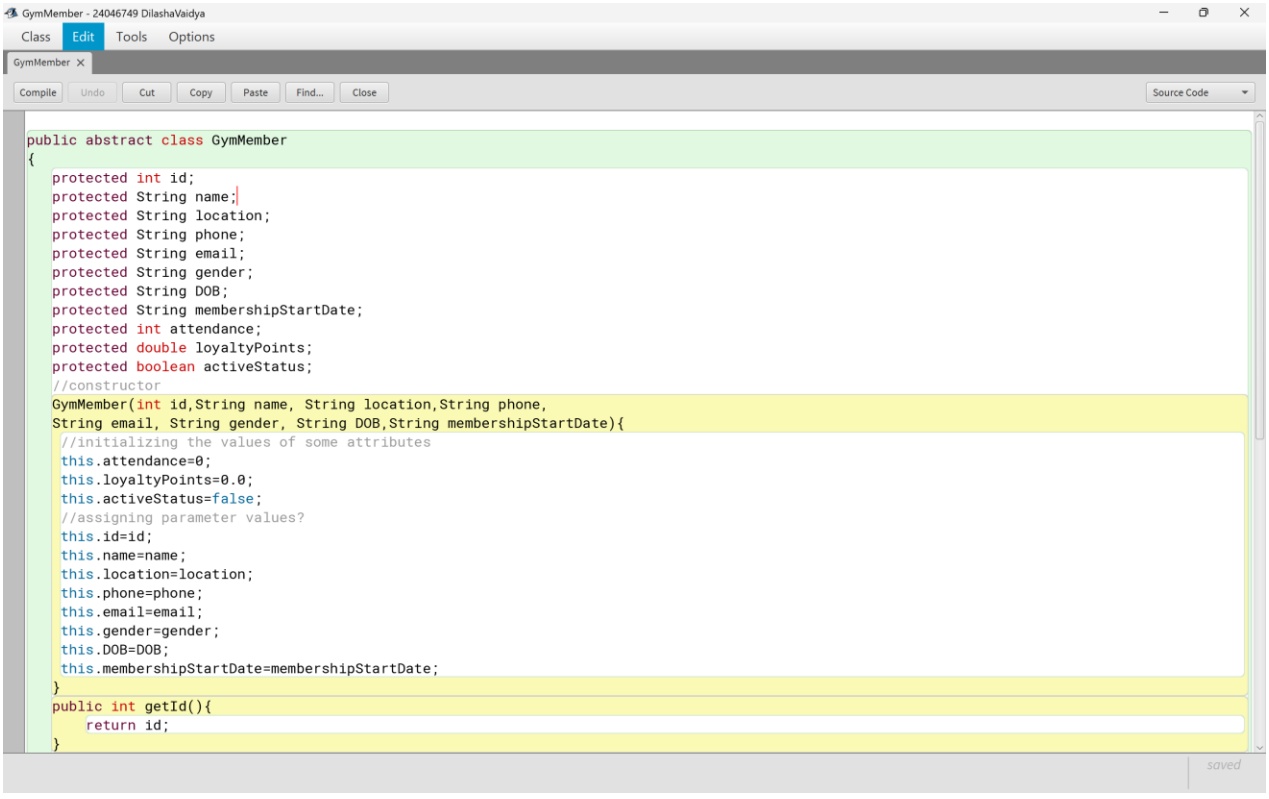


Figure 1: BlueJ interface

Balsamiq – wireframe tool

When designing user interfaces, Balsamiq is a quick wireframing tool that emphasizes clarity and simplicity. For this project, wireframes of the Graphical User Interface (GUI) of the Gym Membership System can be made using Balsamiq prior to implementation. It enables fast GUI layout prototyping of the buttons, input fields, and member management tables. Balsamiq allows for the effective planning of the interface's structure, guaranteeing a design that is both intuitive and easy to use. Visualizing how users will interact with the system is made easier by the tool's drag-and-drop and sketch-style features. This guarantees that the finished Java graphical user interface is orderly and satisfies usability specifications.

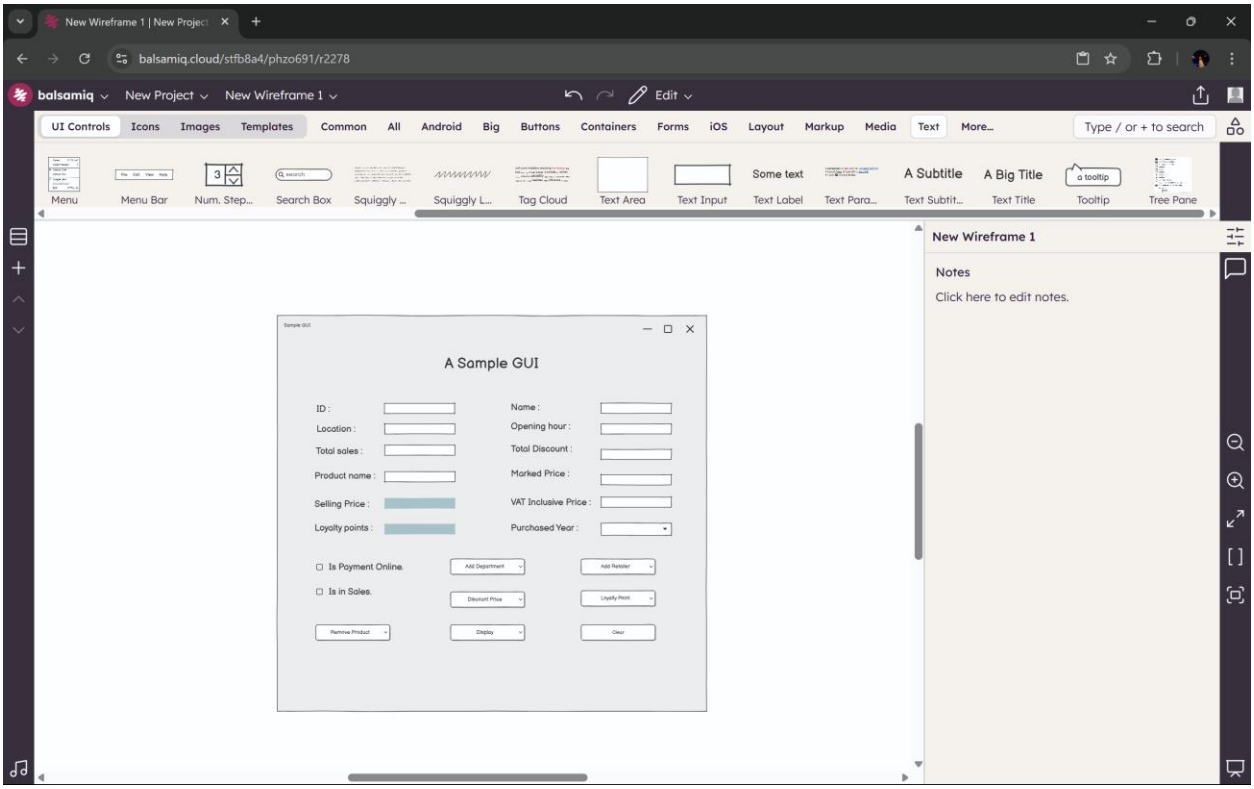


Figure 2: Balsamiq interface

Wireframe

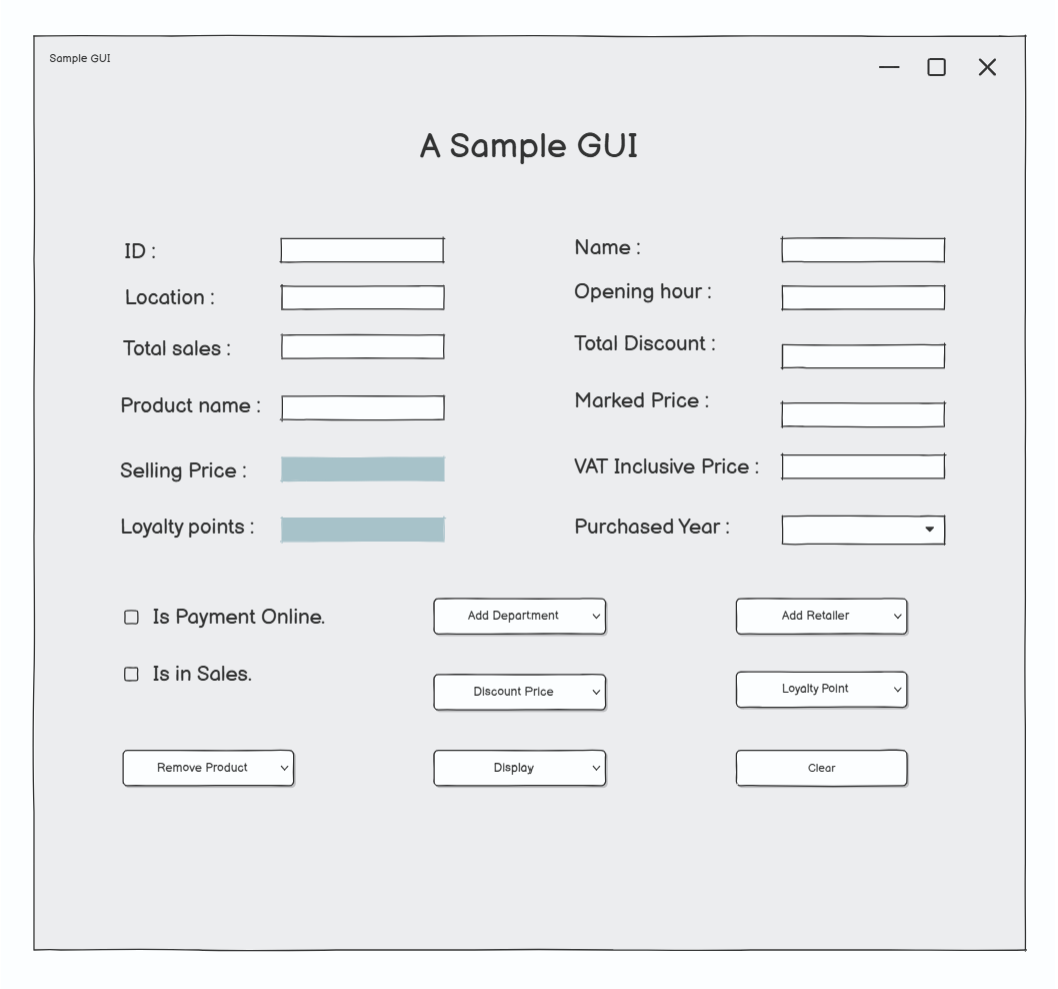


Figure 3: GUI wireframe

GUI

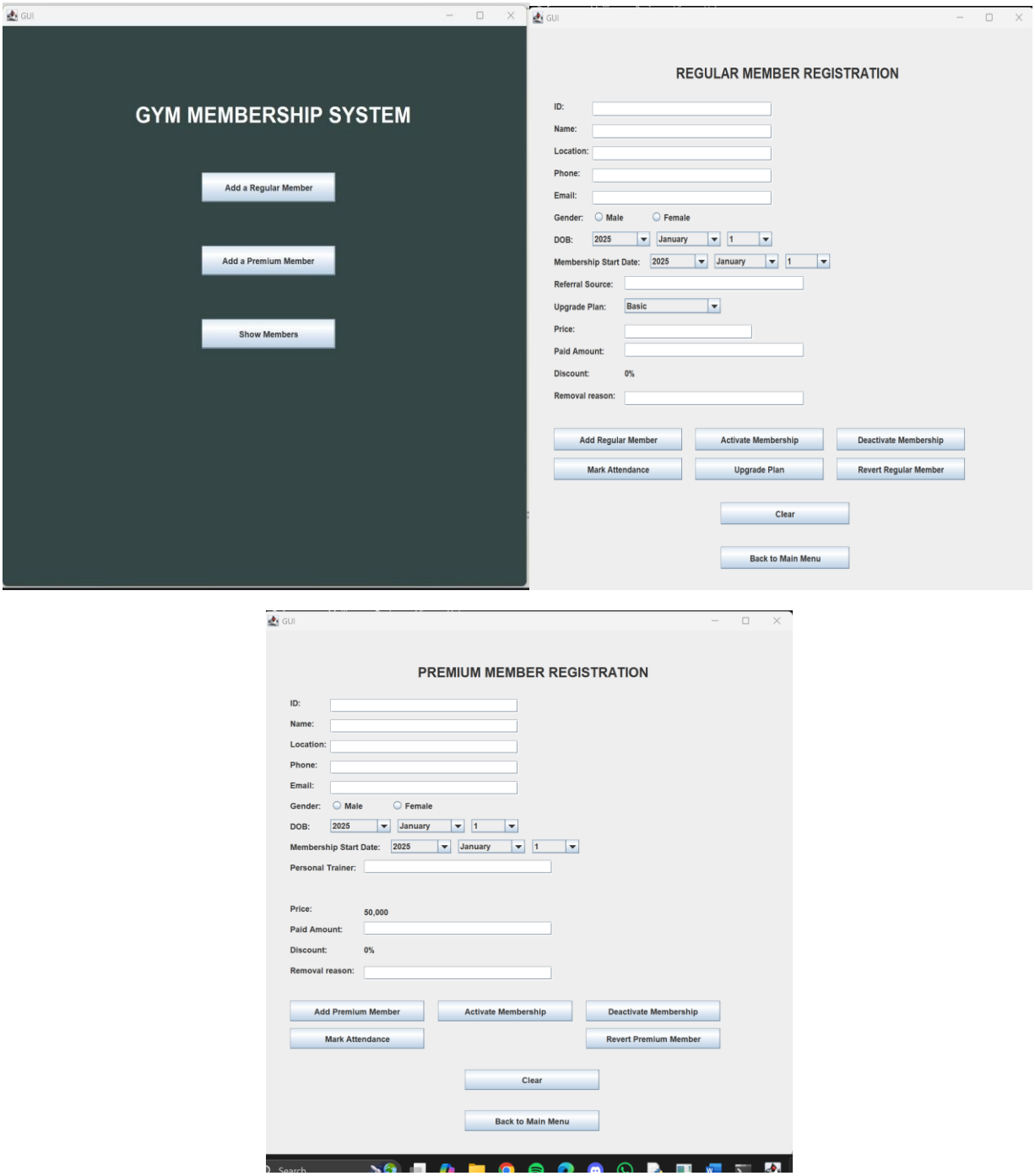
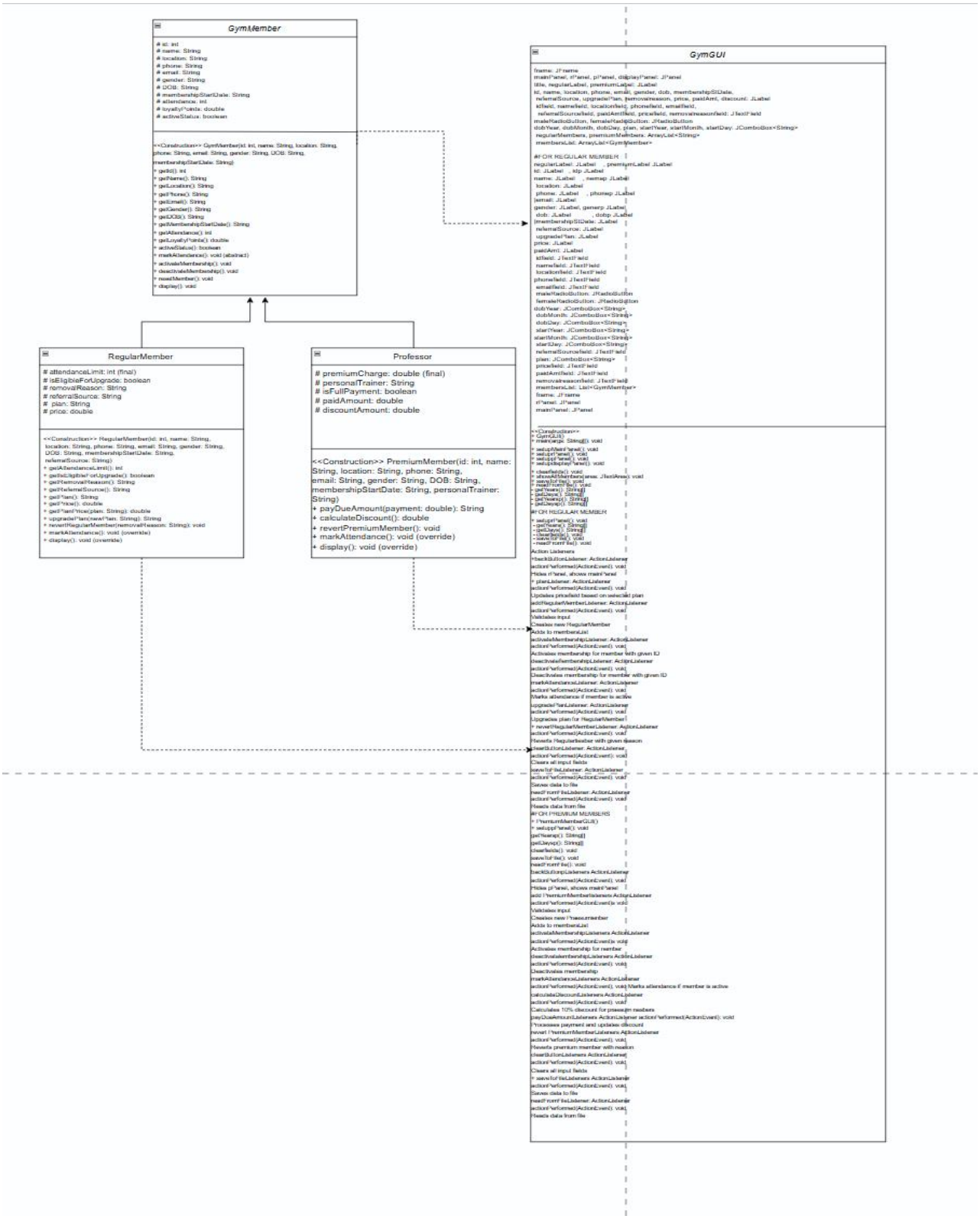


Figure 4: GUI Screenshots

Class Diagram



Pseudocode

Class: GymMember

CREATE an abstract class GymMember

DO

DECLARE an instance variable id as integer using protected access modifier

DECLARE an instance variable name as String using protected access modifier

DECLARE an instance variable location as String using protected access modifier

DECLARE an instance variable phone as String using protected access modifier

DECLARE an instance variable email as String using protected access modifier

DECLARE an instance variable gender as String using protected access modifier

DECLARE an instance variable DOB as String using protected access modifier

DECLARE an instance variable membershipStartDate as String using protected access modifier

DECLARE an instance variable attendance as integer using protected access modifier

DECLARE an instance variable loyaltyPoints as double using protected access modifier

DECLARE an instance variable activeStatus as boolean using protected access modifier

CREATE constructor GymMember with parameters (id, name, location, phone, email, gender, DOB, membershipStartDate)

DO

SET this.attendance to 0

SET this.loyaltyPoints to 0.0

SET this.activeStatus to false

SET this.id to id

SET this.name to name

SET this.location to location

SET this.phone to phone

SET this.email to email

SET this.gender to gender

SET this.DOB to DOB

SET this.membershipStartDate to membershipStartDate

END DO

```
CREATE accessor method getId() with return type integer
DO
   RETURN id
END DO

CREATE accessor method getName() with return type string
DO
   RETURN name
END DO

CREATE accessor method location() with return type string
DO
   RETURN location
END DO

CREATE accessor method phone() with return type string
DO
   RETURN phone
END DO

CREATE accessor method email() with return type string
DO
   RETURN email
END DO

CREATE accessor method gender() with return type string
DO
   RETURN gender
END DO

CREATE accessor method DOB() with return type string
DO
   RETURN DOB
END DO
```

```
CREATE accessor method membershipStartDate() with return type string
DO
   RETURN membershipStartDate
END DO

CREATE accessor method attendance() with return type integer
DO
   RETURN attendance
END DO

CREATE accessor method loyaltyPoints() with return type double
DO
   RETURN loyaltyPoints
END DO

CREATE accessor method activeStatus() with return type boolean
DO
   RETURN activeStatus
END DO

DECLARE abstract method markAttendance() with no return type

CREATE method activateMembership() with no return type
DO
   SET activeStatus to true
END DO

CREATE method deactivateMembership() with no return type
DO
   IF activeStatus is true THEN
      SET activeStatus to false
   ELSE
      DISPLAY "Already inactive"
   END IF
END DO
```

```
    CREATE method resetMember() with no return type
    DO
        SET activeStatus to false
        SET attendance to 0
        SET loyaltyPoints to 0.0
    END DO

    CREATE method display() with no return type
    DO
        DISPLAY "ID: " + id
        DISPLAY "Name: " + name
        DISPLAY "Location: " + location
        DISPLAY "Phone: " + phone
        DISPLAY "Email: " + email
        DISPLAY "Gender: " + gender
        DISPLAY "Date Of Birth: " + DOB
        DISPLAY "Membership startdate: " + membershipStartDate
        DISPLAY "Attendance: " + attendance
        DISPLAY "Loyalty points: " + loyaltyPoints
        DISPLAY "Active Status: " + (IF activeStatus THEN "Active" ELSE "Inactive")
    END DO
END DO
```

Class: RegularMember

CREATE a child class RegularMember that inherits from GymMember

DO

DECLARE a constant instance variable attendanceLimit as integer having protected access modifier

DECLARE a instance variable isEligibleForUpgrade as boolean having protected access modifier

DECLARE a instance variable removalReason as String having protected access modifier

DECLARE a instance variable referralSource as String having protected access modifier

DECLARE a instance variable plan as String having protected access modifier

DECLARE a instance variable price as double having protected access modifier

CREATE constructor RegularMember with parameters (id, name, location, phone, email, gender, DOB, membershipStartDate, referralSource)

DO

CALL parent constructor with (id, name, location, phone, email, gender, DOB, membershipStartDate)

SET this.isEligibleForUpgrade to false

SET this.attendanceLimit to 30

SET this.plan to "Basic"

SET this.price to 6500.0

SET this.removalReason to " "

SET this.referralSource to referralSource

END DO

CREATE accessor method getAttendanceLimit() with return type integer

DO

RETURN attendanceLimit

END DO

CREATE accessor method getIsEligibleForUpgrade() with return type boolean

DO

RETURN isEligibleForUpgrade

END DO

CREATE accessor method getRemovalReason() with return type string

DO

RETURN removalReason

END DO

```
CREATE accessor method getReferralSource() with return type string
DO
   RETURN referralSource
END DO

CREATE accessor method getPlan() with return type string
DO
   RETURN plan
END DO

CREATE accessor method getPrice() with return type double
DO
   RETURN price
END DO

CREATE method getPlanPrice(plan) with return type double
DO
   CONVERT plan to lowercase

   IF plan equals "basic" THEN
      RETURN 6500.0
   ELSE IF plan equals "standard" THEN
      RETURN 12500.0
   ELSE IF plan equals "deluxe" THEN
      RETURN 18500.0
   ELSE
      RETURN -1
   END IF
END DO

CREATE method upgradePlan(newPlan) with return type string
DO
   IF attendance is greater than or equal to attendanceLimit THEN
      SET isEligibleForUpgrade to true
   END IF
```

```
    IF isEligibleForUpgrade is false THEN
        RETURN "Member is not eligible for an upgrade"
    END IF

    IF plan equals newPlan (case-insensitive) THEN
        RETURN "You are already subscribed to this plan"
    END IF

    COMPUTE newPrice as getPlanPrice(newPlan)

    IF newPrice equals -1 THEN
        RETURN "invalid plan selected"
    END IF

    SET this.plan to newPlan
    SET this.price to newPrice
    RETURN "Plan successfully upgraded to " + newPlan + " for " + newPrice
END DO


CREATE method revertRegularMember(removalReason) with no return type
DO
    CALL resetMember()
    SET this.isEligibleForUpgrade to false
    SET this.plan to "Basic"
    SET this.price to 6500.0
    SET this.removalReason to removalReason
END DO

OVERRIDE method markAttendance() with no return type
DO
    IF attendance is less than attendanceLimit THEN
        INCREMENT attendance by 1
        ADD 5 to loyaltyPoints
```

```
        ELSE
            DISPLAY "Attendance limit reached for regular member."
        END IF
    END DO

    OVERRIDE method display() with no return type
    DO
        CALL parent display method
        DISPLAY "Plan: " + plan
        DISPLAY "Price: " + price

        IF removalReason is not empty THEN
            DISPLAY "Removal reason: " + removalReason
        END IF
    END DO
END DO
```

Class: PremiumMember
CREATE a class PremiumMember that inherits from GymMember

DO

DECLARE final attribute premiumCharge as double and SET to 50000

DECLARE attribute personalTrainer as string

DECLARE attribute isFullPayment as boolean and SET to false

DECLARE attribute paidAmount as double and SET to 0

DECLARE attribute discountAmount as double and SET to 0

CREATE constructor PremiumMember with parameters (id, name, location, phone, email, gender, DOB, membershipStartDate, personalTrainer)

DO

CALL super constructor with parameters (id, name, location, phone, email, gender, DOB, membershipStartDate)

SET this.personalTrainer to personalTrainer

END DO

CREATE method payDueAmount with parameter (payment) with return type string

DO

IF isFullPayment is true THEN

RETURN "Payment is already full"

END IF

ADD payment to paidAmount

IF paidAmount is greater than premiumCharge THEN

RETURN "Payment exceeds the premium charge"

END IF

IF paidAmount equals premiumCharge THEN

SET isFullPayment to true

END IF

COMPUTE remainingAmount as premiumCharge minus paidAmount

RETURN "payment successful. Remaining amount:" + remainingAmount

END DO

```
    CREATE method discount with no return type
    DO
        IF isFullPayment is true THEN
            COMPUTE discountAmount as premiumCharge multiplied by 0.10
            DISPLAY "Discount given:" + discountAmount
        END IF
    END DO

    CREATE method revertPremiumMember with no return type
    DO
        CALL super.resetMember()
        SET this.personalTrainer to " "
        SET this.isFullPayment to false
        SET this.paidAmount to 0
        SET this.discountAmount to 0
    END DO

    OVERRIDE method markAttendance with no return type
    DO
    END DO

    OVERRIDE method display with no return type
    DO
        CALL super.display()
        DISPLAY "Personal trainer:" + personalTrainer
        DISPLAY "Paid amount:" + paidAmount
        DISPLAY "Full payment:" + isFullPayment
        COMPUTE remainingAmount as premiumCharge minus paidAmount
        DISPLAY "remaining amount:" + remainingAmount

        IF isFullPayment is true THEN
            DISPLAY "discounf amount:" + discountAmount
        END IF
    END DO
END DO
```

Class: GymGUI

IMPORT GUI components (JFrame, JPanel, JButton, JLabel, JTextField, JComboBox, JRadioButton, ButtonGroup, JOptionPane, JTextArea, JScrollPane)

IMPORT Layout components (BorderLayout)

IMPORT Event handling (ActionListener, ActionEvent)

IMPORT File handling (FileWriter, BufferedWriter, FileReader, BufferedReader, IOException)

IMPORT Utility components (Font, Color, ArrayList)

CREATE class GymGUI

DO

DECLARE frame as JFrame

DECLARE mainPanel, rPanel, pPanel, displayPanel as JPanel

DECLARE various labels (title, regularLabel, id, name, etc.) as JLabel

DECLARE and INITIALIZE idfield, namefield, locationfield, phonefield, emailfield, etc. as JTextField

DECLARE maleRadioButton, femaleRadioButton, maleRadioButtonp, femaleRadioButtonp as JRadioButton

DECLARE dobYear, dobMonth, dobDay, plan, startYear, startMonth, startDay, etc. as JComboBox<String>

DECLARE regularMembers, premiumMembers as ArrayList<String>

DECLARE membersList as ArrayList<GymMember>

CREATE main method with parameter (args as string array)

DO

CREATE new GymGUI()

END DO

CREATE method clearfields() with no return type

DO

SET idfield text to ""

SET namefield text to ""

SET locationfield text to ""

SET phonefield text to ""

SET emailfield text to ""

SET referralSourcefield text to ""

SET paidAmtfield text to ""

```
    SET dobYear selection to index 0
    SET dobMonth selection to index 0
    SET dobDay selection to index 0
    SET startYear selection to index 0
    SET startMonth selection to index 0
    SET startDay selection to index 0

    SET idpfield text to ""
    SET namepfield text to ""
    SET locationpfield text to ""
    SET phonepfield text to ""
    SET emailpfield text to ""
    SET paidAmtpfield text to ""
    SET removalreasonpfield text to ""
    SET personalTrainerpfield text to ""
    SET dobYearp selection to index 0
    SET dobMonthp selection to index 0
    SET dobDayp selection to index 0
    SET startYearp selection to index 0
    SET startMonthp selection to index 0
    SET startDayp selection to index 0
END DO

CREATE constructor GymGUI() with no parameters
DO
    INITIALIZE regularMembers as new ArrayList
    INITIALIZE premiumMembers as new ArrayList
    INITIALIZE membersList as new ArrayList

    INITIALIZE frame as JFrame with title "GUI"
    SET frame size to 800x800
    SET frame default close operation to EXIT_ON_CLOSE
    SET frame layout to null

    INITIALIZE mainPanel as JPanel
```

```
    SET mainPanel bounds to 0, 0, 800, 800
    SET mainPanel layout to null
    SET mainPanel visibility to true

    INITIALIZE rPanel as JPanel
    SET rPanel bounds to 0, 0, 800, 800
    SET rPanel layout to null
    SET rPanel visibility to false

    INITIALIZE pPanel as JPanel
    SET pPanel bounds to 0, 0, 800, 800
    SET pPanel layout to null
    SET pPanel visibility to false

    INITIALIZE displayPanel as JPanel
    SET displayPanel bounds to 0, 0, 800, 800
    SET displayPanel layout to null
    SET displayPanel visibility to false

    ADD mainPanel to frame
    ADD rPanel to frame
    ADD pPanel to frame
    ADD displayPanel to frame

    CALL setupMainPanel()
    CALL setuprPanel()
    CALL setuppPanel()
    CALL setupdisplayPanel()

    SET frame visibility to true
END DO

CREATE method setupMainPanel() with no return type
DO
    SET mainPanel background color to dark teal (52,70,70)
```

INITIALIZE title as JLabel with text "GYM MEMBERSHIP SYSTEM"

SET title foreground color to white

SET title font to Montserrat, BOLD, size 30

INITIALIZE addRegularButton as JButton with text "Add a Regular Member"

INITIALIZE addPremiumButton as JButton with text "Add a Premium Member"

INITIALIZE showMembersButton as JButton with text "Show Members"

SET title bounds to 200, 80, 500, 80

ADD title to mainPanel

SET addRegularButton bounds to 300, 200, 200, 40

ADD addRegularButton to mainPanel

SET addPremiumButton bounds to 300, 300, 200, 40

ADD addPremiumButton to mainPanel

SET showMembersButton bounds to 300, 400, 200, 40

ADD showMembersButton to mainPanel

ADD ActionListener to addRegularButton that performs:

DO

  SET mainPanel visibility to false

  SET rPanel visibility to true

  SET pPanel visibility to false

  SET displayPanel visibility to false

END DO

ADD ActionListener to addPremiumButton that performs:

DO

  SET mainPanel visibility to false

  SET rPanel visibility to false

  SET pPanel visibility to true

  SET displayPanel visibility to false

```
    END DO

    ADD ActionListener to showMembersButton that performs:
    DO
        SET mainPanel visibility to false
        SET rPanel visibility to false
        SET pPanel visibility to false
        SET displayPanel visibility to true
    END DO
END DO

CREATE method setuprPanel() with no return type
DO
    INITIALIZE regularLabel as JLabel with text "REGULAR MEMBER REGISTRATION"
    SET regularLabel font to Montserrat, BOLD, size 20
    SET regularLabel bounds to 230, 20, 400, 80
    ADD regularLabel to rPanel

    CREATE and POSITION form elements (labels, text fields, radio buttons, etc.)
    ADD them to rPanel

    CALL getYears() and STORE result in years
    DECLARE months array with all month names
    CALL getDays() and STORE result in days

    CREATE ComboBoxes for dates (dobYear, dobMonth, dobDay, startYear, startMonth, startDay)
    ADD them to rPanel

    INITIALIZE various action buttons (add, activate, deactivate, mark attendance, etc.)
    ADD them to rPanel

    INITIALIZE backButton as JButton with text "Back to Main Menu"
    SET backButton bounds to 300, 700, 200, 30
    ADD backButton to rPanel
```

```
ADD ActionListener to backButton that performs:
DO
   SET rPanel visibility to false
   SET mainPanel visibility to true
END DO

ADD ActionListener to plan combo box that performs:
DO
   GET selected plan
   IF selected plan equals "Basic" THEN
      SET pricefield text to "6500"
   ELSE IF selected plan equals "Standard" THEN
      SET pricefield text to "12500"
   ELSE IF selected plan equals "Deluxe" THEN
      SET pricefield text to "18500"
   END IF
END DO

ADD ActionListener to addRegularMemberButton that performs:
DO
   TRY
      CONVERT idfield text to integer and STORE in id
      GET values from other form fields

      // Check for duplicate ID
      FOR each member in membersList
      DO
         IF member id equals id THEN
            DISPLAY message "Member ID already exists!"
            RETURN
         END IF
      END DO

      CONCATENATE selected date values for DOB and membership start date
```

```
            CREATE RegularMember with form values
            ADD to membersList

            DISPLAY message "Regular Member added successfully!"
            CALL clearfields()
        CATCH NumberFormatException
            DISPLAY message "Please enter valid numeric values for ID."
        CATCH other exceptions
            DISPLAY message with exception details
        END TRY
    END DO

END DO

CREATE method getYears() with return type string array
DO
    DECLARE years as string array of size 100
    SET currentYear to 2025

    FOR i from 0 to years.length-1
    DO
        SET years[i] to string value of (currentYear - i)
    END DO

    RETURN years
END DO

CREATE method getDays() with return type string array
DO
    DECLARE days as string array of size 31

    FOR i from 0 to days.length-1
    DO
        SET days[i] to string value of (i + 1)
    END DO
```

```
    RETURN days
END DO

CREATE method setuppPanel() with no return type
DO
    INITIALIZE premiumLabel as JLabel with text "PREMIUM MEMBER REGISTRATION"
    SET premiumLabel font to Montserrat, BOLD, size 20
    SET premiumLabel bounds to 230, 20, 400, 80
    ADD premiumLabel to pPanel

END DO

CREATE method getYearsp() with return type string array
DO
    DECLARE years as string array of size 100
    SET currentYear to 2025

    FOR i from 0 to years.length-1
    DO
        SET years[i] to string value of (currentYear - i)
    END DO

    RETURN years
END DO

CREATE method getDaysp() with return type string array
DO
    DECLARE days as string array of size 31

    FOR i from 0 to days.length-1
    DO
        SET days[i] to string value of (i + 1)
    END DO
```

```
    RETURN days
END DO

CREATE method setupdisplayPanel() with no return type
DO
    INITIALIZE title as JLabel with text "All Gym Members"
    SET title bounds to 300, 20, 200, 30
    ADD title to displayPanel

    INITIALIZE membersArea as JTextArea
    SET membersArea editable to false
    INITIALIZE scroll as JScrollPane containing membersArea
    SET scroll bounds to 50, 60, 700, 600
    ADD scroll to displayPanel

    INITIALIZE refreshBtn as JButton with text "Refresh"
    SET refreshBtn bounds to 300, 680, 100, 30
    ADD refreshBtn to displayPanel

    INITIALIZE saveToFileButtond as JButton with text "Save to File"
    SET saveToFileButtond bounds to 200, 680, 100, 30
    ADD saveToFileButtond to displayPanel

    INITIALIZE readFromFileButtond as JButton with text "Read from File"
    SET readFromFileButtond bounds to 550, 680, 100, 30
    ADD readFromFileButtond to displayPanel

    INITIALIZE backBtn as JButton with text "Back"
    SET backBtn bounds to 420, 680, 100, 30
    ADD backBtn to displayPanel

    CALL showAllMembers(membersArea)

    ADD ActionListener to saveToFileButtond that performs:
    DO
```

```
        CALL saveToFile()
    END DO

    ADD ActionListener to readFromFileButtond that performs:
    DO
        CALL readFromFile()
    END DO
END DO

CREATE method showAllMembers(area as JTextArea) with no return type
DO
    INITIALIZE StringBuilder sb

    IF membersList is empty THEN
        APPEND "No members registered yet.\n" to sb
    ELSE
        APPEND "Regular Members\n" to sb

        FOR each member in membersList
        DO
            IF member is instance of RegularMember THEN
                CAST member to RegularMember as rm
                APPEND member details to sb
            END IF
        END DO

        APPEND "Premium Members\n" to sb

        FOR each member in membersList
        DO
            IF member is instance of PremiumMember THEN
                CAST member to PremiumMember as pm
                APPEND member details to sb
            END IF
        END DO
```

```
    END IF

    SET area text to sb string value
END DO

CREATE method saveToFile() with no return type
DO
    TRY
        INITIALIZE FileWriter for "members.txt"
        INITIALIZE BufferedWriter with fileWriter

        WRITE formatted header to bufferedWriter

        FOR each member in membersList
        DO
            IF member is instance of RegularMember THEN
                CAST member to RegularMember as rm
                WRITE formatted regular member details to bufferedWriter
            ELSE IF member is instance of PremiumMember THEN
                CAST member to PremiumMember as pm
                WRITE formatted premium member details to bufferedWriter
            END IF
        END DO

        CLOSE bufferedWriter
        DISPLAY message "Member details saved to members.txt successfully!"
    CATCH IOException
        DISPLAY message with error details
    END TRY
END DO

CREATE method readFromFile() with no return type
DO
    INITIALIZE readFrame as JFrame with title "Members from File"
    SET readFrame size to 800x600
```

```
        SET readFrame layout to BorderLayout

        INITIALIZE textArea as JTextArea
        SET textArea editable to false
        INITIALIZE scrollPane as JScrollPane containing textArea

        TRY
            INITIALIZE FileReader for "members.txt"
            INITIALIZE BufferedReader with fileReader

            DECLARE line as string
            WHILE (read line from bufferedReader is not null)
            DO
                APPEND line + newline to textArea
            END DO

            CLOSE bufferedReader
        CATCH IOException
            SET textArea text to error message
        END TRY

        ADD scrollPane to center of readFrame

        INITIALIZE closeButton as JButton with text "Close"
        ADD ActionListener to closeButton that performs:
        DO
            DISPOSE readFrame
        END DO

        INITIALIZE buttonPanel as JPanel
        ADD closeButton to buttonPanel
        ADD buttonPanel to south of readFrame
        SET readFrame visibility to true
    END DO
END DO
```

Method Description

Class: GymMember

Accessor Methods (Getters):

- getId(): Returns the member's ID number.
- getName(): Returns the member's name.
- location(): Returns the member's location.
- phone(): Returns the member's phone number.
- email(): Returns the member's email address.
- gender(): Returns the member's gender.
- DOB(): Returns the member's date of birth.
- membershipStartDate(): Returns the date when the member's gym membership began.
- attendance(): Returns the member's current attendance count.
- loyaltyPoints(): Returns the member's accumulated loyalty points.
- activeStatus(): Returns whether the member's membership is currently active (true) or inactive (false).

Abstract Methods

- markAttendance(): An abstract method that must be implemented by subclasses to track member attendance.
- Membership Management Methods
- activateMembership(): Sets the member's status to active.
- deactivateMembership(): Sets the member's status to inactive if currently active; displays a message if already inactive.
- resetMember(): Resets the member's data by setting active status to false, attendance to 0, and loyalty points to 0.0.

Display Method

- display(): Prints all member information to the console, including ID, name, location, contact details, membership information, attendance, loyalty points, and active status.

Class: RegularMember

Accessor Methods

- getAttendanceLimit(): Returns the maximum number of allowed attendances for a regular member.
- getIsEligibleForUpgrade(): Returns a boolean indicating whether the member is eligible for a plan upgrade.
- getRemovalReason(): Returns the reason for removal or reversion of a member, if applicable.
- getReferralSource(): Returns the source that referred this member to the gym.
- getPlan(): Returns the current membership plan of the regular member.
- getPrice(): Returns the current price of the member's plan.

Plan Management Methods

- getPlanPrice(String plan): Returns the price for a specified plan type. Returns -1 if the plan is invalid.
- upgradePlan(String newPlan): Attempts to upgrade the member's plan to the specified new plan. Checks eligibility, validates the new plan, and updates the member's plan and price if all conditions are met. Returns a status message.
- revertRegularMember(String removalReason): Resets the member to default state, sets the plan back to "Basic", price to default, and records the reason for reversion.

Overridden Methods

- markAttendance(): Overrides the parent class method to increment attendance and loyalty points if the attendance limit has not been reached. Prints a message if the limit is reached.
- display(): Overrides the parent class method to display all member information including the additional RegularMember attributes such as plan, price, and removal reason (if applicable).

Class: PremiumMember

Accessor Methods

- getPersonalTrainer(): Returns the name of the member's personal trainer.
- getIsFullPayment(): Returns a boolean indicating whether the member has made a full payment.
- getPaidAmount(): Returns the amount the member has paid so far.
- getDiscountAmount(): Returns the discount amount applied to the member's payment.

Payment Methods

- payDueAmount(double payment): Processes a payment from the member. If the payment is at least 50,000, applies a 10% discount and marks the payment as complete. If less than 50,000, records it as a partial payment. Returns a message detailing the payment status.
- calculateDiscount(): Calculates and displays the discount amount (10% of the premium charge) if full payment has been made.

Member Management Methods

- revertPremiumMember(): Resets the member to default state by calling the parent class's resetMember() method and resetting all PremiumMember-specific attributes.
- markAttendance(): Overrides the parent class's markAttendance method (implementation is empty in this code).
- display(): Overrides the parent class's display method to show all member information including personal trainer, payment status, remaining amount, and discount (if applicable).
- RetryClaude can make mistakes. Please double-check responses.

Class: GymGUI

Main Methods

- main(String[] args): The entry point of the application that creates a new instance of GymGUI.
- GymGUI(): Constructor that initializes the GUI components, sets up panels, and makes the frame visible.

Setup Methods

- setupMainPanel(): Configures the main menu panel with buttons for adding regular members, premium members, and displaying all members.
- setuprPanel(): Sets up the panel for regular member registration and management, including form fields and buttons for various actions.
- setuppPanel(): Configures the panel for premium member registration and management, similar to the regular member panel but with premium-specific fields.
- setupdisplayPanel(): Creates a panel to display all gym members with options to refresh the view and navigate back to the main menu.

Utility Methods

- clearfields(): Resets all input fields and dropdown selections to their default values.
- getYears(): Creates an array of years (from current year back 100 years) for date selection dropdowns in the regular member panel.
- getDays(): Generates an array of days (1-31) for date selection dropdowns in the regular member panel.
- getYearsp(): Same as getYears() but specifically for the premium member panel.
- getDaysp(): Same as getDays() but specifically for the premium member panel.

Data Management Methods

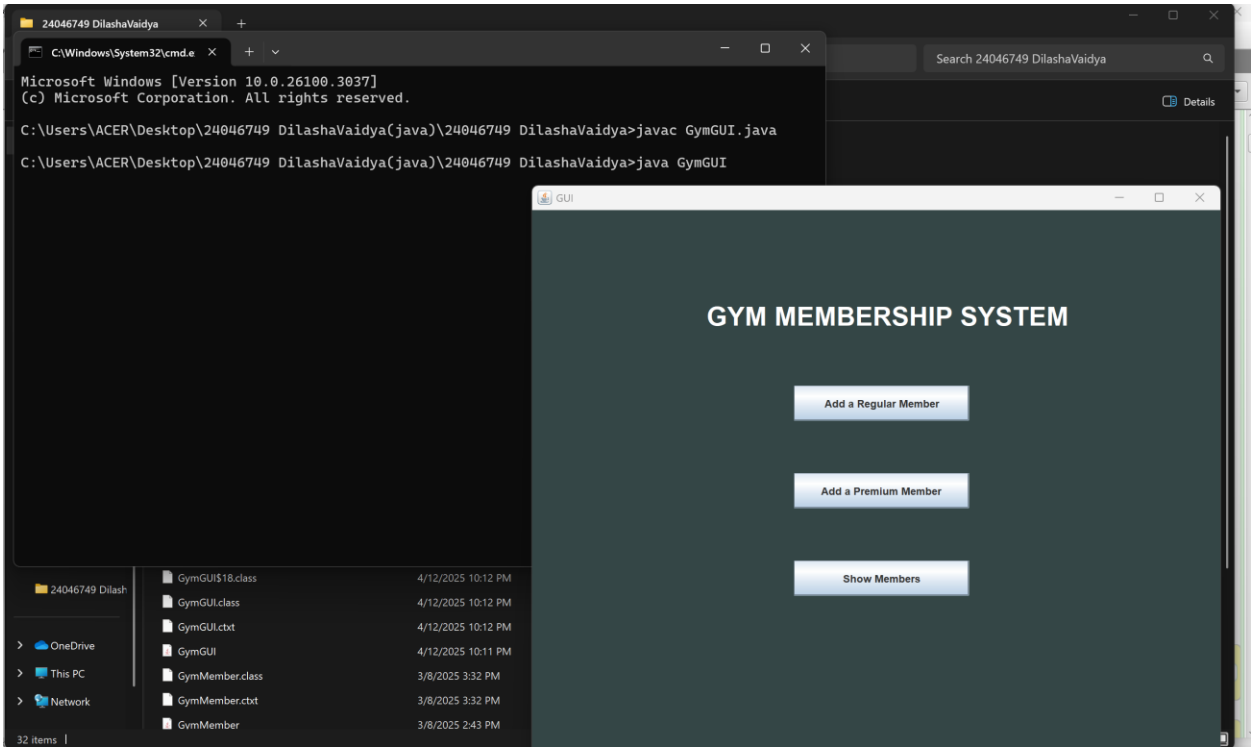
- showAllMembers(JTextArea area): Populates a text area with information about all gym members, categorized by membership type.
- saveToFile(): Exports all member data to a text file named "members.txt" in a formatted tabular structure.
- readFromFile(): Reads member data from the "members.txt" file and displays it in a new window with a scrollable text area

Testing

Test 1: Compile and run the program using command prompt/terminal

Table 1: Test 1

Objective	To compile and run the program using the command prompt/terminal
Action	The following command was typed in the command terminal: Javac GymGUI.java Java GymGUI
Expected output	The program should compile, run and the GUI screen should be displayed.
Actual Output	The program was compiled, run and the GUI screen was displayed.
Result	The test was successful.



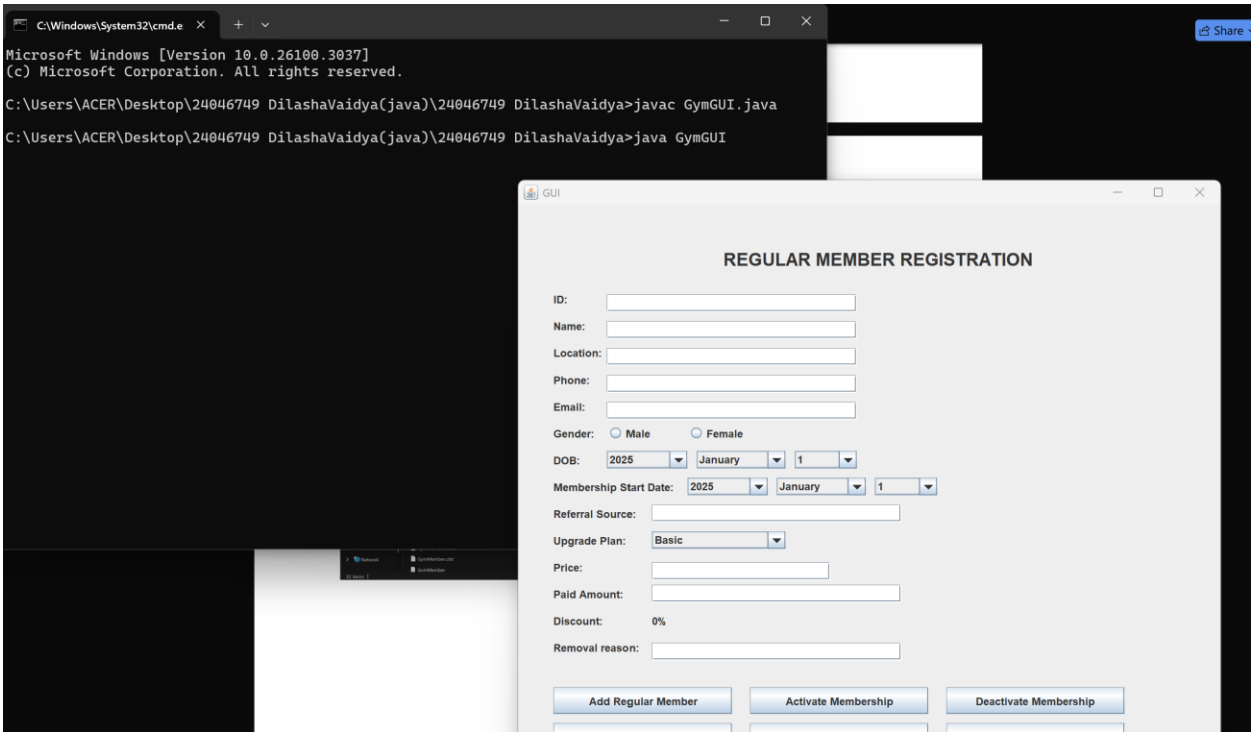
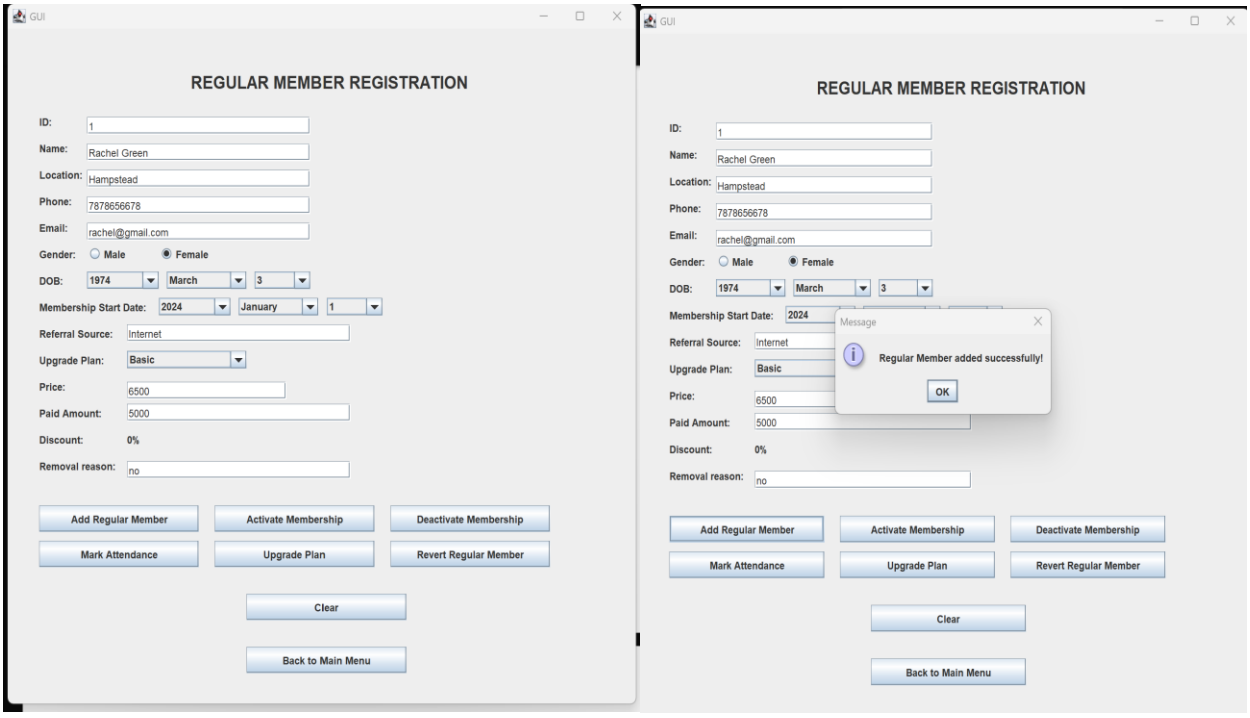


Figure 5: Test 1: Program run through command prompt

Test 2: Adding regular member and premium member respectively

Table 2: Test 2

Objective	To add a regular and premium member respectively
Action	All the required fields were filled and ‘Add regular member’/’Add premium member’ was clicked.
Expected output	The regular/premium member details should be added in a dialog box.
Actual Output	The regular/premium member details were stored in a dialog box.
Result	The test was successful.



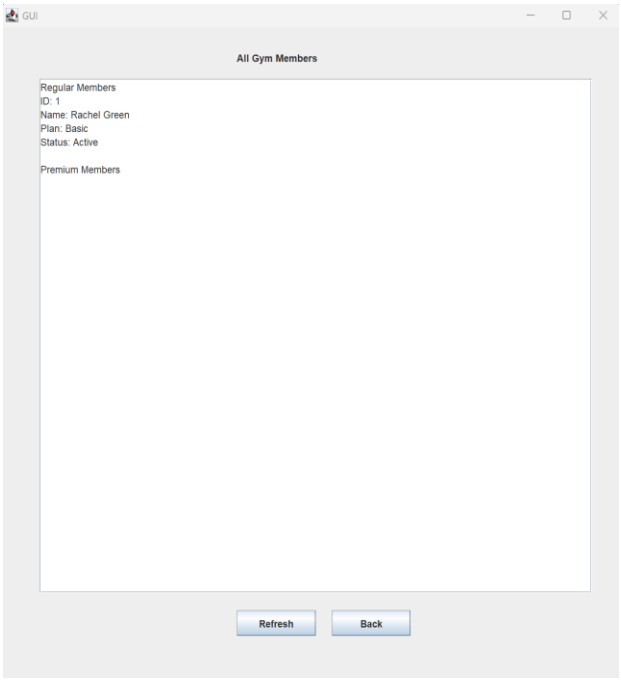
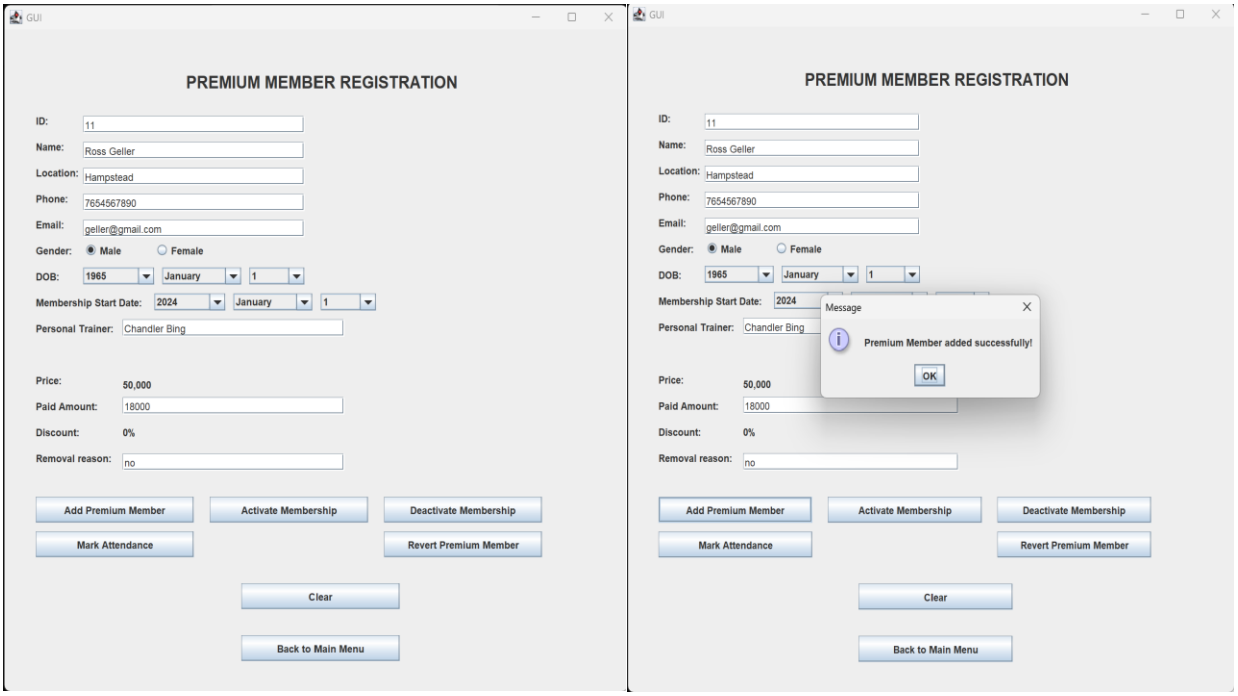


Figure 6: Test 2: Adding regular member



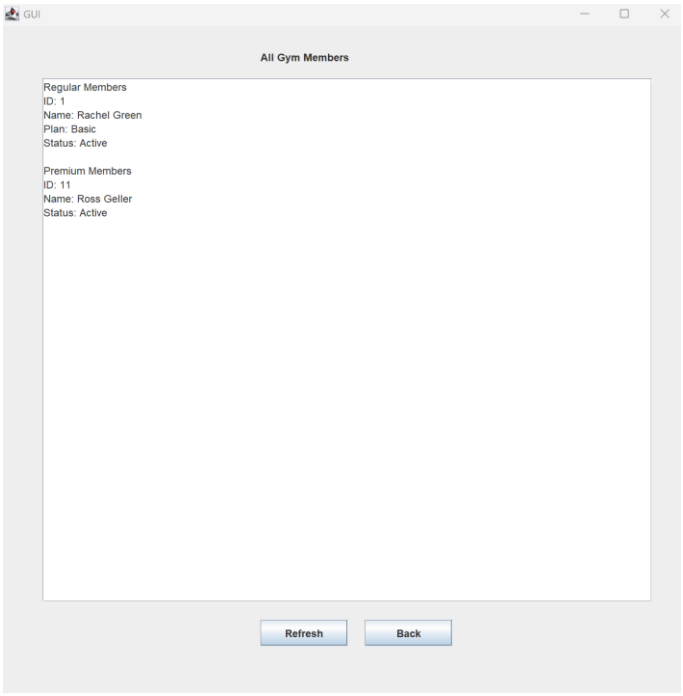


Figure 7: Test 2: adding premium member

Test 3: Test Case for Mark Attendance and upgrade plan

Table 3: Test 3

Objective	To test whether attendance is marked from the button.
Action	The member Id was given and attendance was marked.
Expected output	A message dialog box for ‘successful attendance marking’ should be displayed.
Actual Output	A message dialog box for attendance marking was shown.
Result	The test was successful.

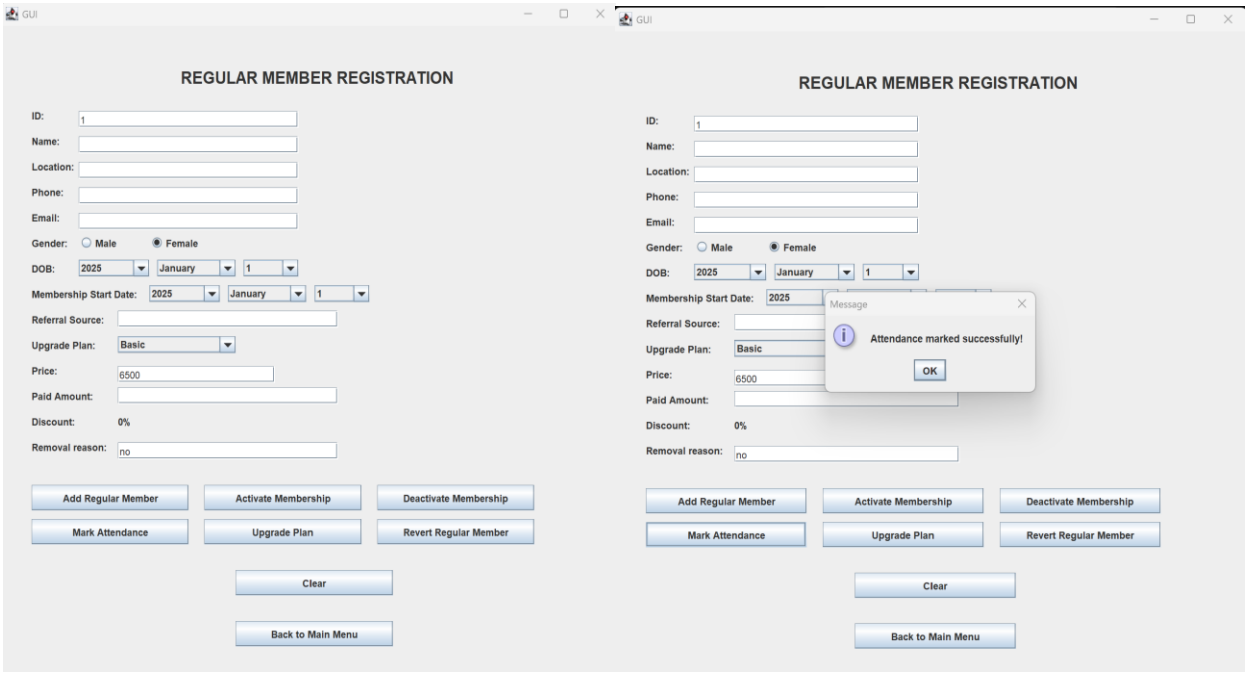
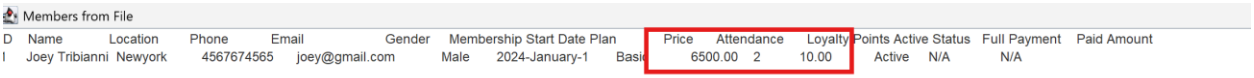
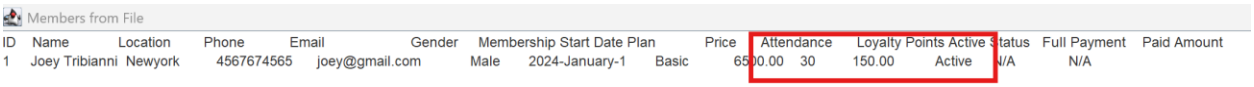


Figure 8: Test 3: Mark attendance testing

Table 4: Test 3 (2)

Objective	To upgrade the gym membership plan of the user
Action	The membership plan of the user was upgraded in terms of the conditions.
Expected output	The program should upgrade the membership plan of the user and an appropriate message dialog box should appear.
Actual Output	The program upgraded the plan of the user and a message dialog box for the upgrade was shown.
Result	The test was successful.





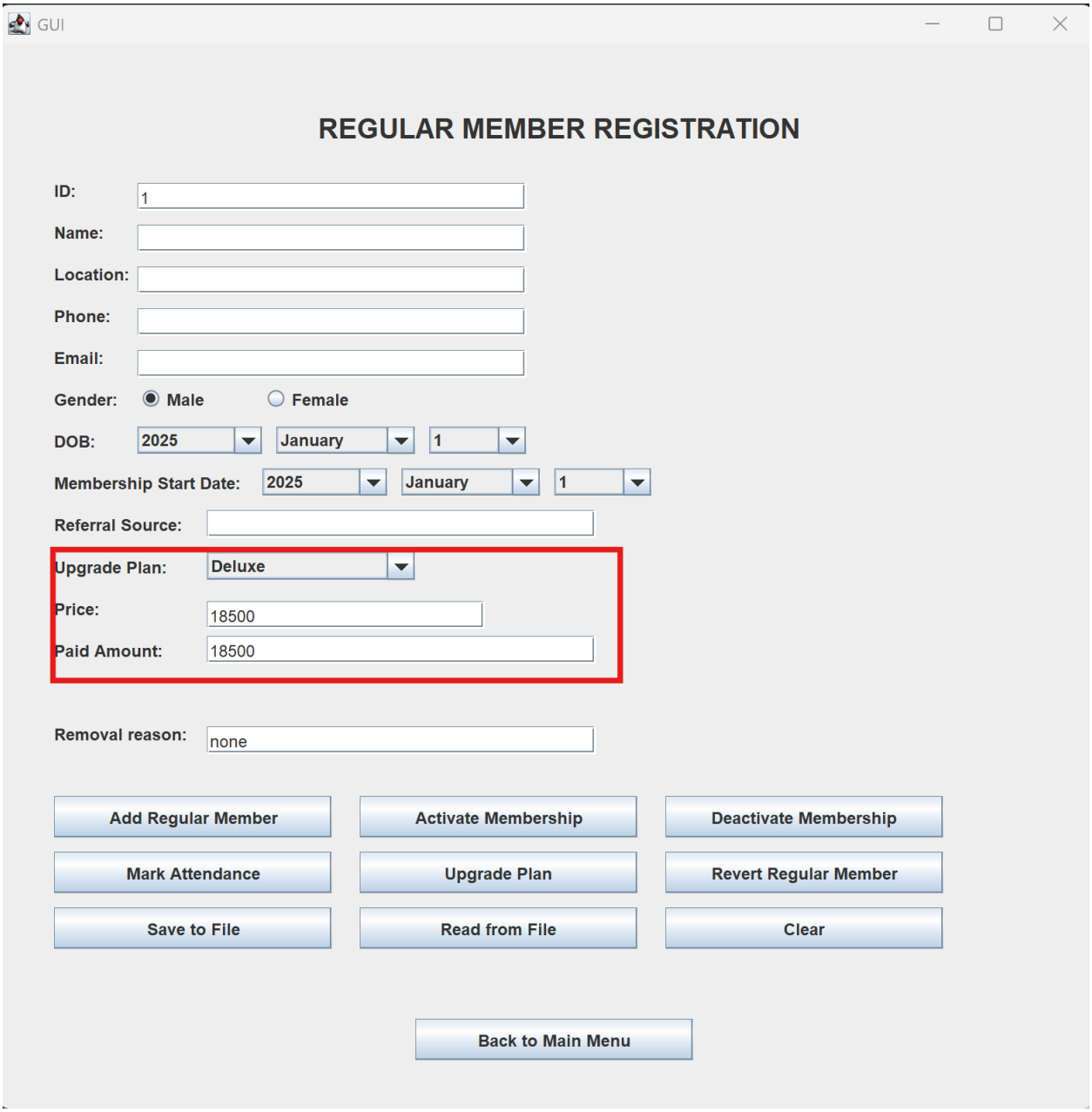
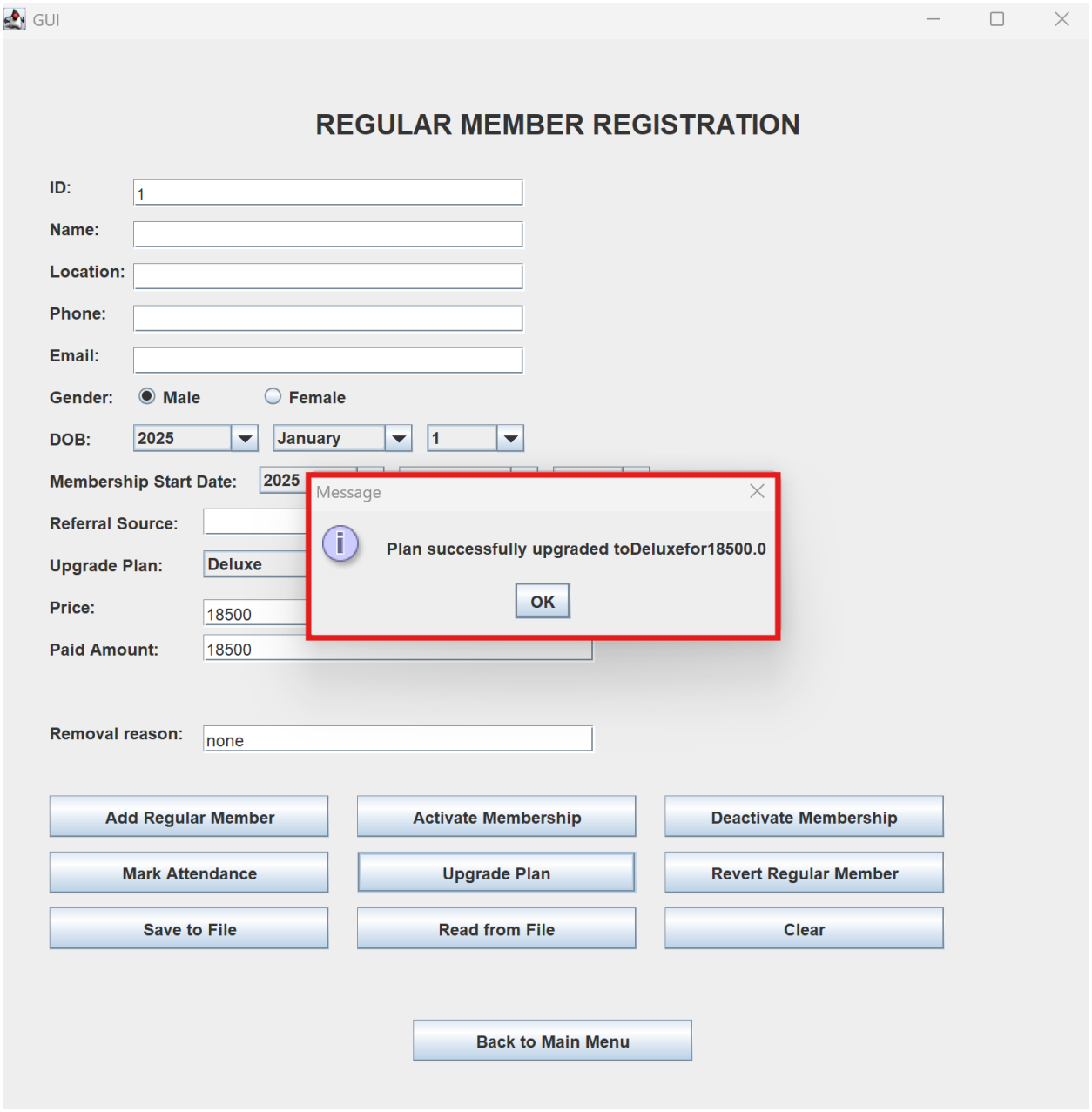


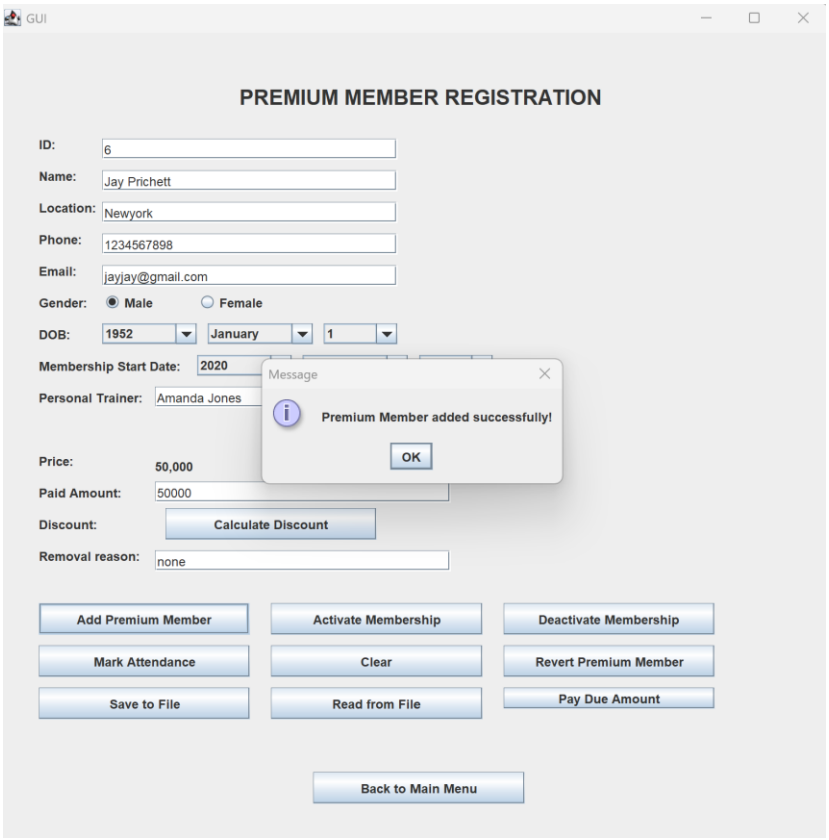
Figure 9: Test 3: Upgrade plan

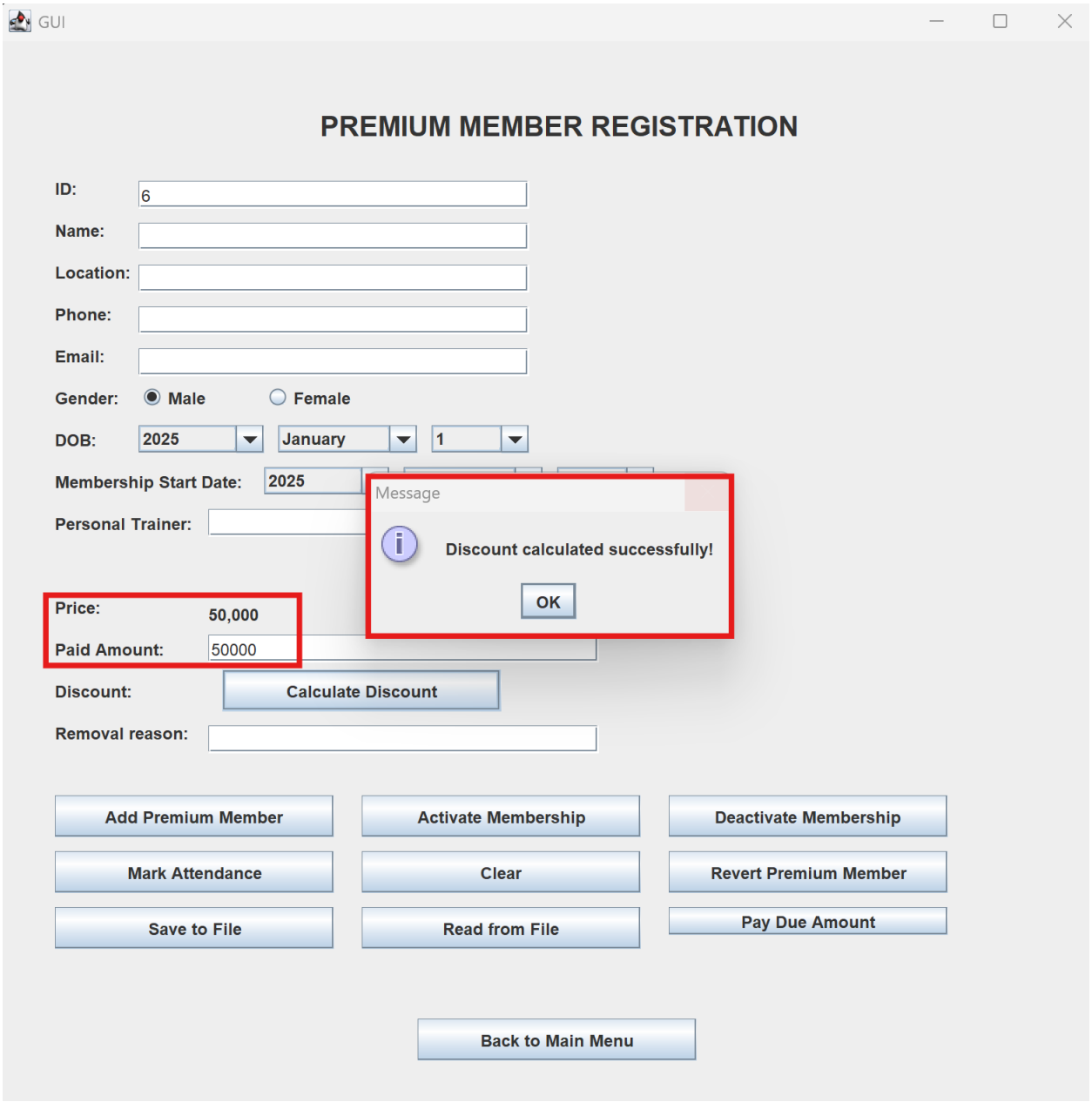

GUI
REGULAR MEMBER REGISTRATION
ID:
1
Name:
Location:
Phone:
Email:
Gender:
Male
Female
DOB:
2025
January
1
Membership Start Date:
2025
Referral Source:
Upgrade Plan:
Deluxe
Price:
18500
Paid Amount:
18500
Message
Plan successfully upgraded toDeluxefor18500.0
OK
Removal reason:
none
Add Regular Member
Activate Membership
Deactivate Membership
Mark Attendance
Upgrade Plan
Revert Regular Member
Save to File
Read from File
Clear
Back to Main Menu

Test 4: Calculate discount, pay due, revert member

Table 5: Test 4

Objective	To calculate discount in case of full payment
Action	Pull payment was put in the payment textbox and discount was calculated
Expected output	10% discount should be given to the user and a message box should be displayed.
Actual Output	10% discount was calculated and shown
Result	The test was successful.




GUI
PREMIUM MEMBER REGISTRATION
ID:
6
Name:
Location:
Phone:
Email:
Gender:
Male
Female
DOB:
2025
January
1
Membership Start Date:
2025
Personal Trainer:
Message
Discount calculated successfully!
OK
Price:
50,000
Paid Amount:
50000
Discount:
Calculate Discount
Removal reason:
Add Premium Member
Activate Membership
Deactivate Membership
Mark Attendance
Clear
Revert Premium Member
Save to File
Read from File
Pay Due Amount
Back to Main Menu

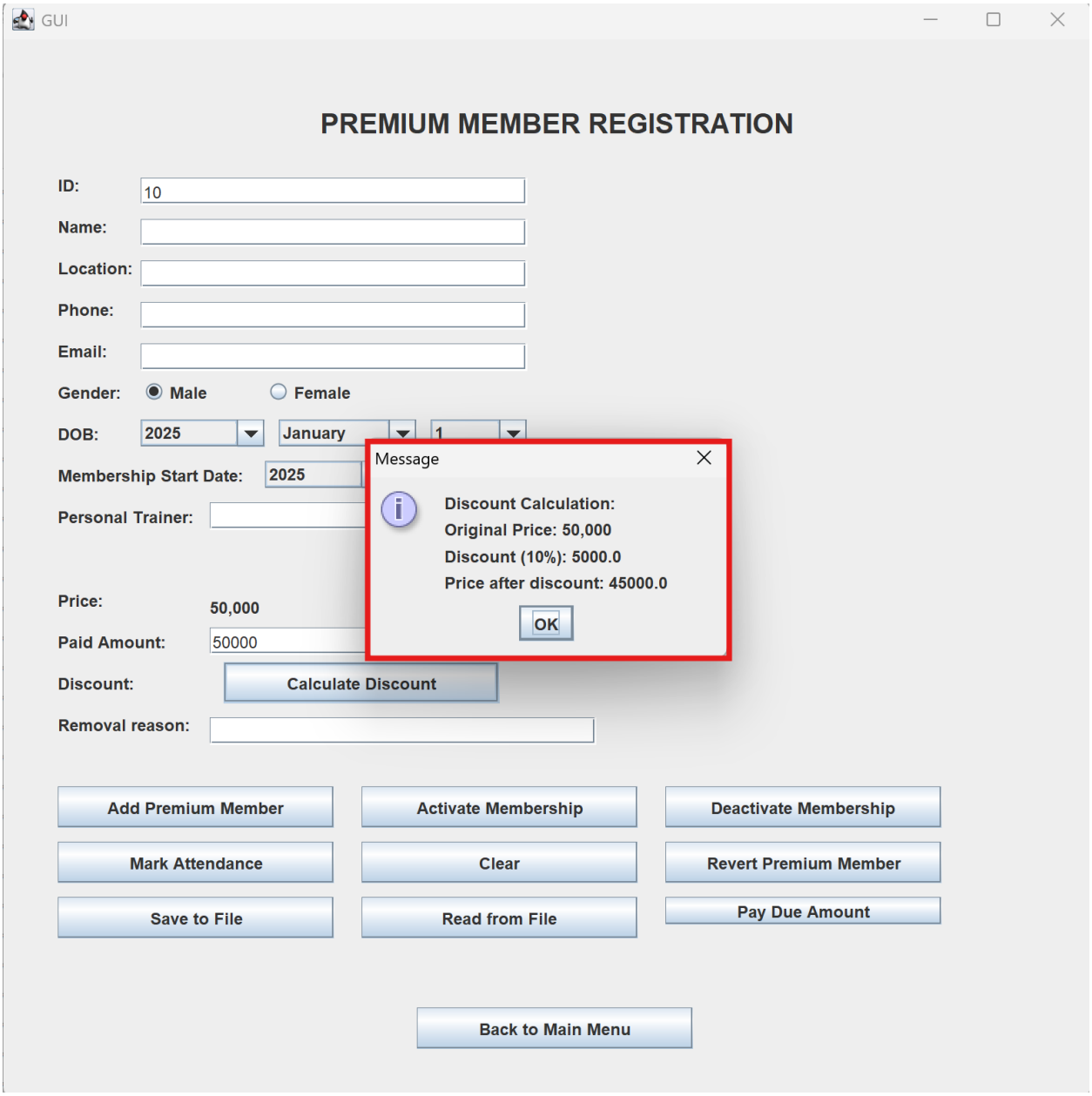


Figure 10: Test 5: Discount calculation

Table 6: Test 5 (2)

Objective	To pay the due amount left
Action	Incomplete payment was made, due payment was shown and on payment message box was displayed.
Expected output	The program should display the left due amount and on payment an appropriate message box should appear.
Actual Output	The program displayed the due amount and on payment, showed a message box with an appropriate message.
Result	The test was successful.

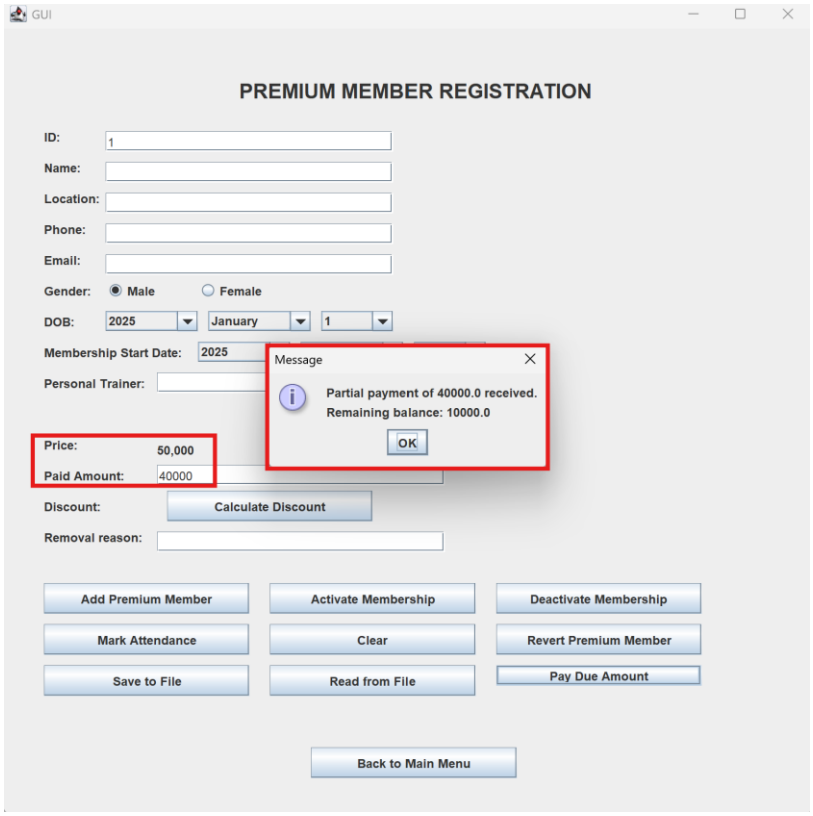


Figure 11: Test 5: Pay due amount

Table 7: Test 5 (3)

Objective	To revert the member
Action	Incomplete payment was made, due payment was shown and on payment message box was displayed.
Expected output	The program should display the left due amount and on payment an appropriate message box should appear.
Actual Output	The program displayed the due amount and on payment, showed an message box with an appropriate message.
Result	The test was successful.

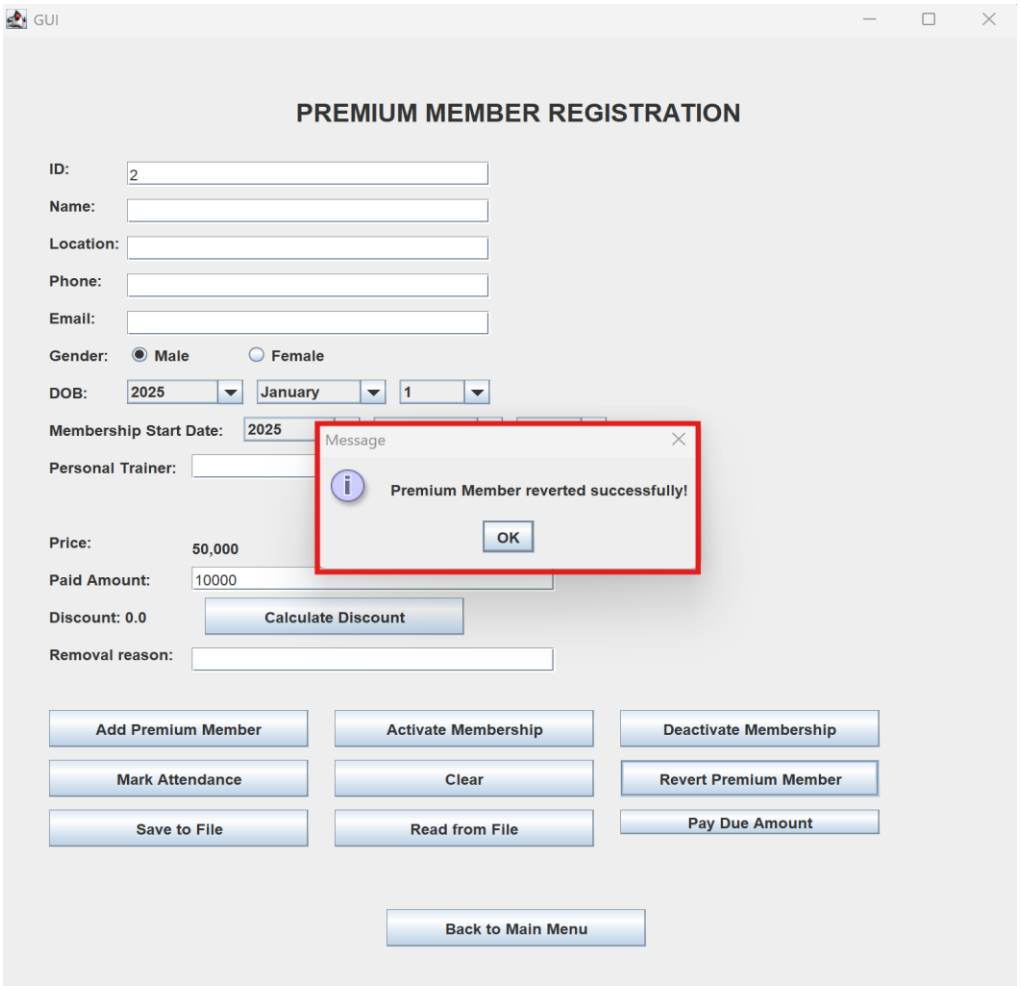


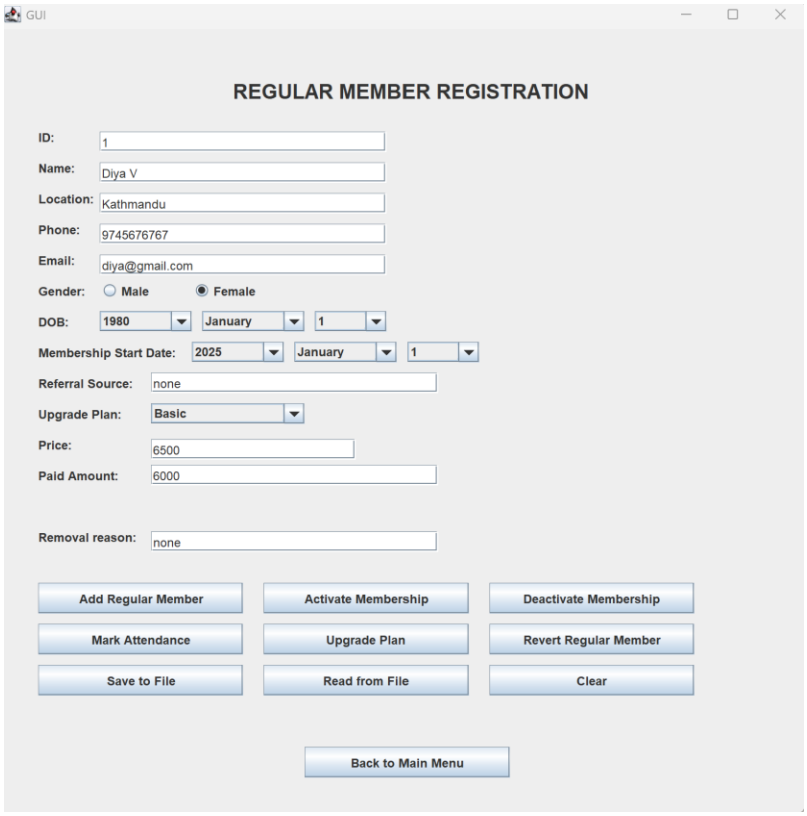
Figure 12: Test 5: Revert membership

Test 5

Save to and read from file:

Table 8: Test 5

Objective	To save and read from file
Action	A member should be registered and was saved to read from the file
Expected output	A member was registered and was saved to read from the file
Actual Output	The data was saved and read from the file
Result	The test was successful.



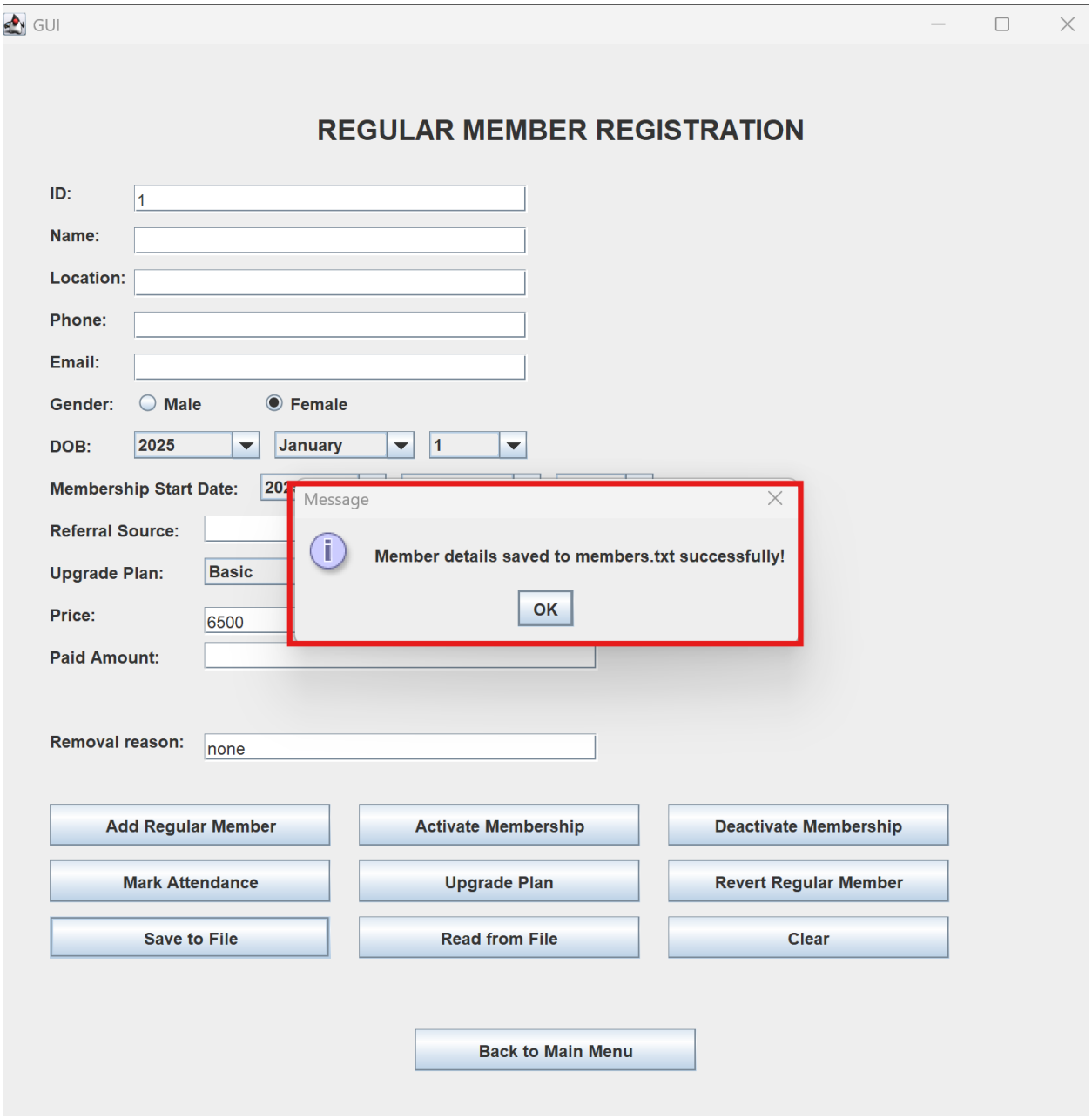


Figure 13: Saved to filee

Figure 14: Read from file

Error Detection and Correction

Syntax Error

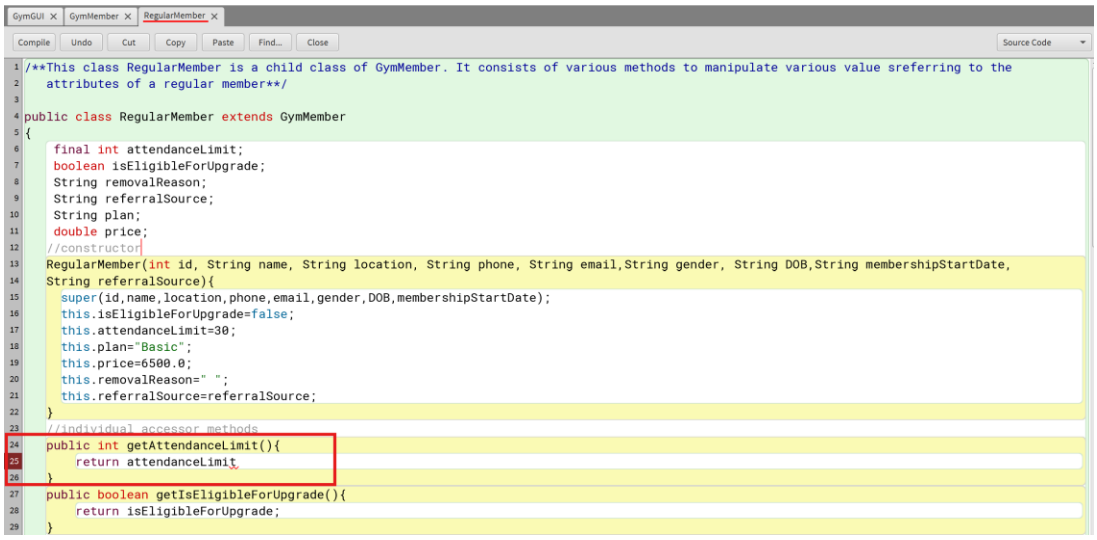


Figure 15: Syntax error

A semicolon was missing on the line of code highlighted red, which was then fixed by adding a semicolon.

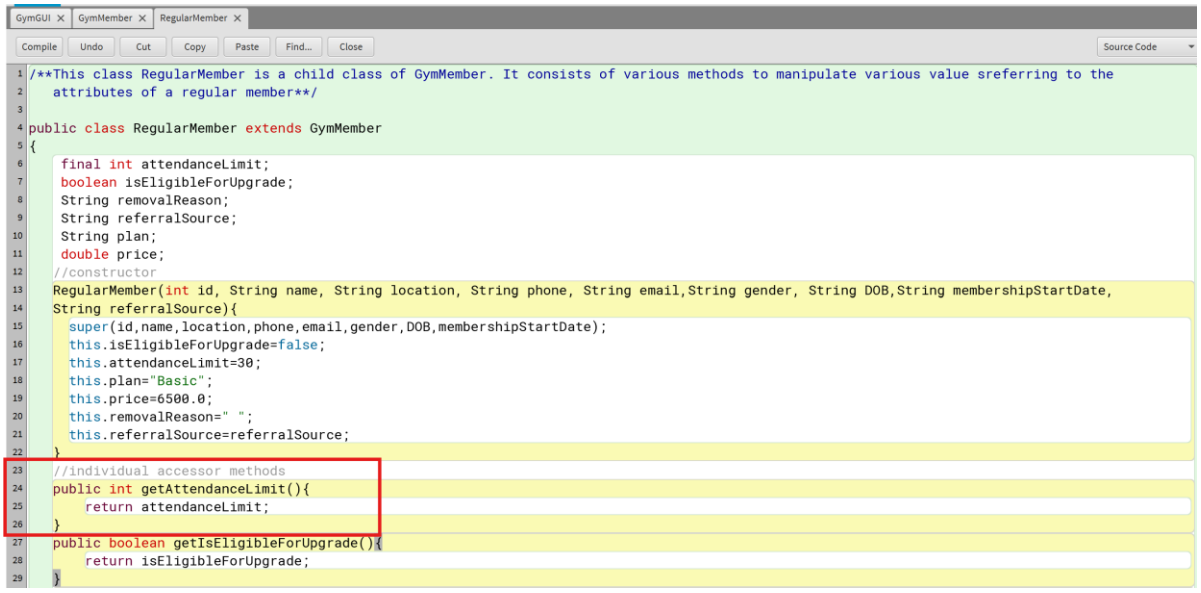


Figure 16: Syntax error correction

Logical Error

Correcting taken inputs of phone number > or < 10 digits:

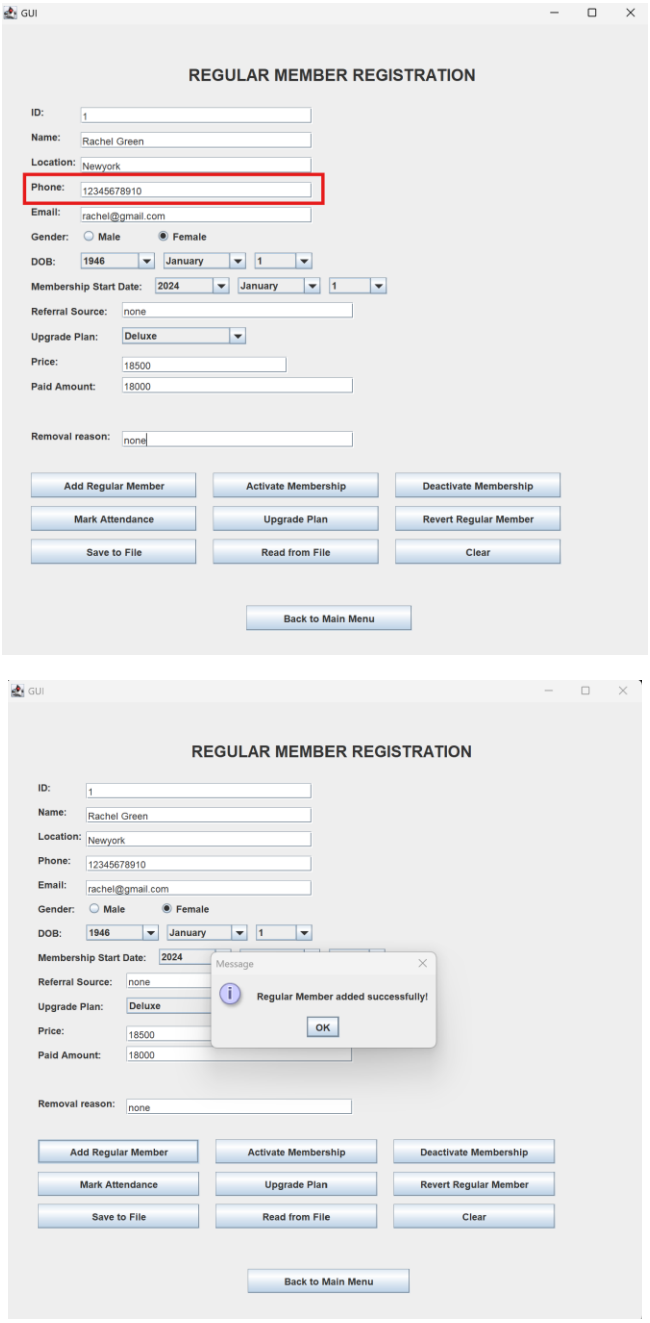


Figure 17: Logical error of taking phone numbers greater than 10 digits

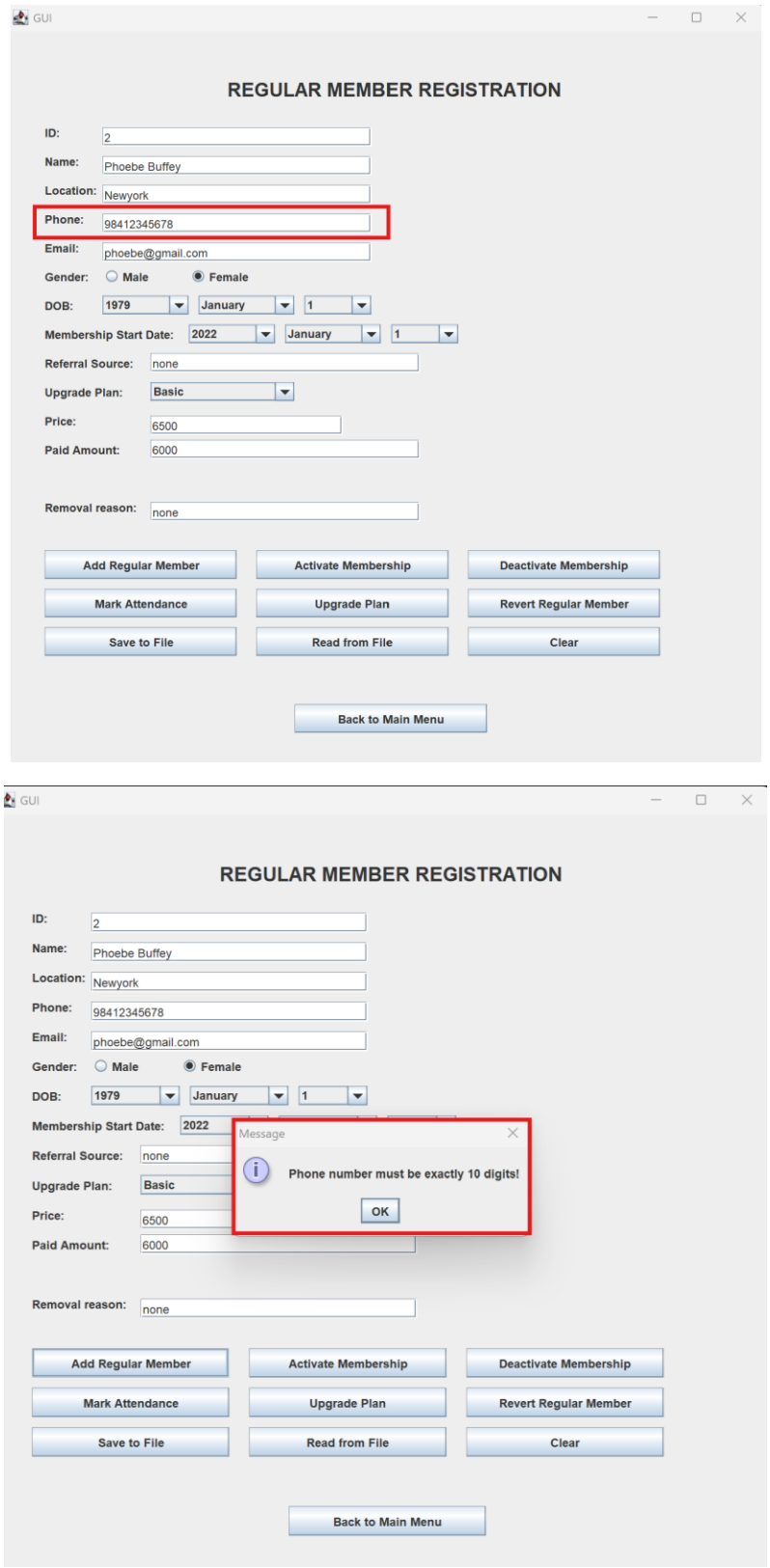
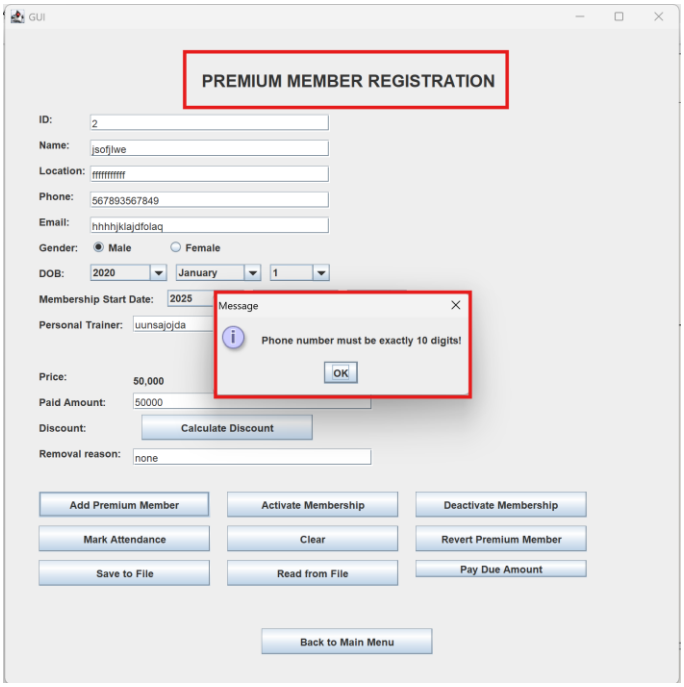


Figure 18: Fixed logical error (1)

```
addRegularMemberButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            int id = Integer.parseInt(idfield.getText());
            String name = namefield.getText();
            String location = locationfield.getText();
            String phone = phonefield.getText();
            String email = emailfield.getText();
            if (phone.length() != 10 || !phone.matches("\\d{10}")) {
                JOptionPane.showMessageDialog(frame, "Phone number must be exactly 10 digits!");
                return;
            }
            String gender = maleRadioButton.isSelected() ? "Male" : "Female";
            //String dob = dobYear.getText();
            //String startDate = membershipStartField.getText();
            String referral = referralSourcefield.getText();

            // Check for duplicate ID
            for (GymMember member : (membersList)){
                if (member.getId() == id) {
                    JOptionPane.showMessageDialog(frame, "Member ID already exists!");
                    return;
                }
            }
```

Figure 19: Improved code to fix the error



Logical error of taken inputs of negative member IDs:

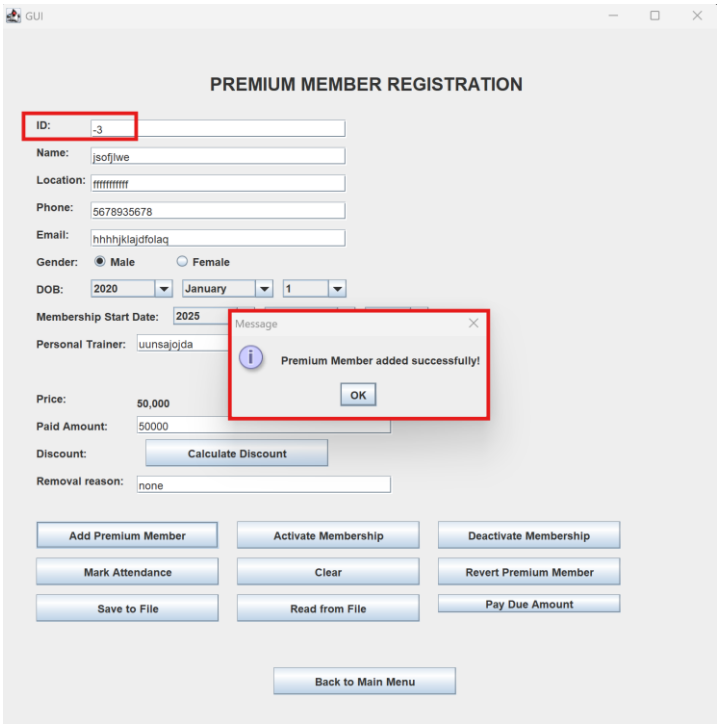


Figure 20: Negative member ID is taken

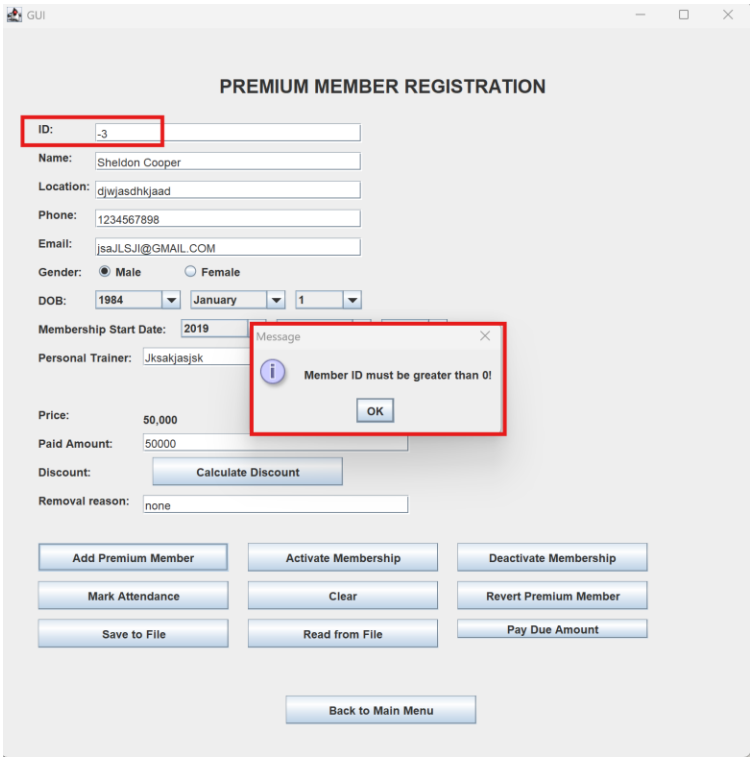
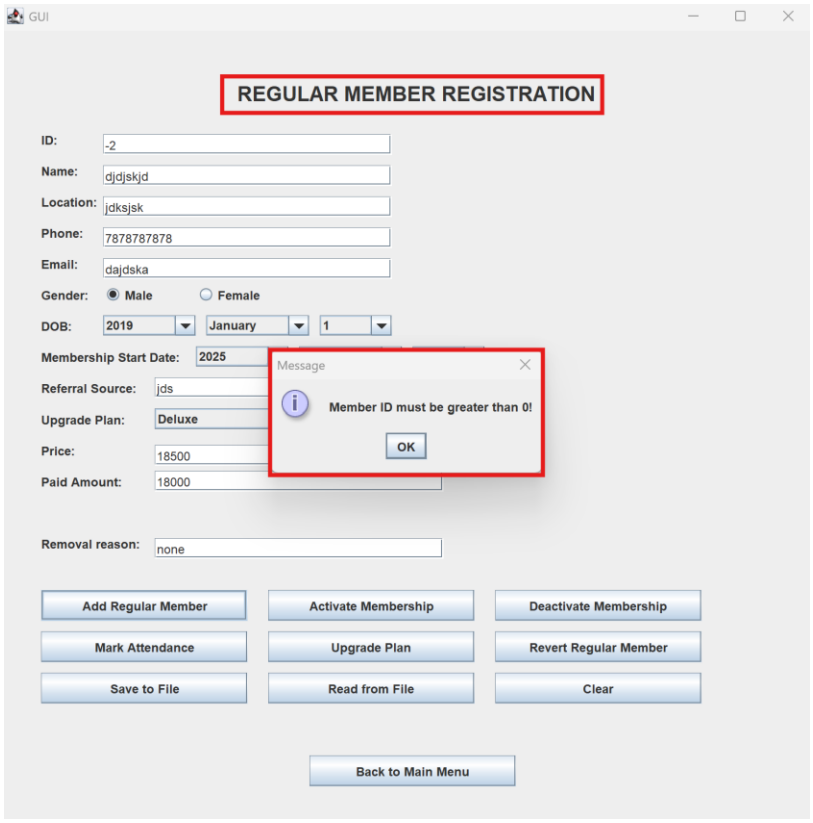


Figure 21: Negative member ID is not taken

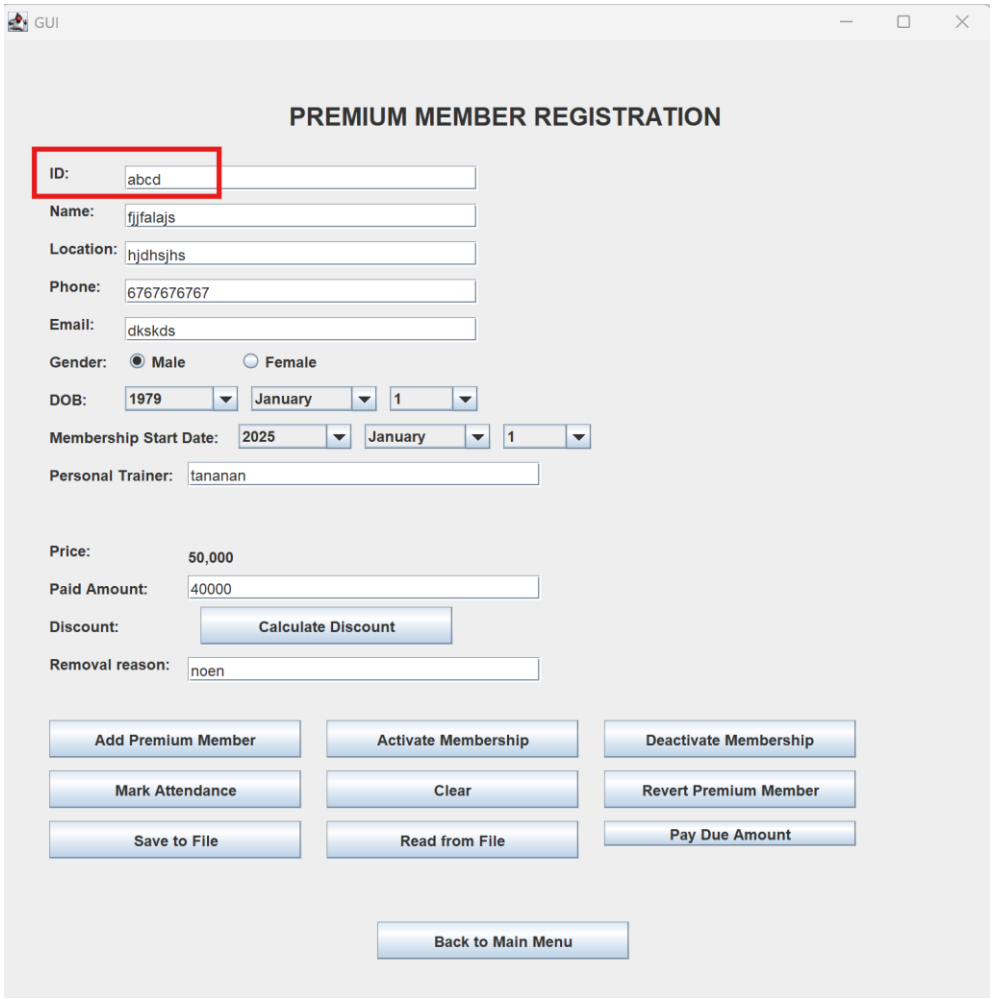



```
addPremiumMemberButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            int idp = Integer.parseInt(idpfield.getText());
            if (idp <= 0) {
            JOptionPane.showMessageDialog(frame, "Member ID must be greater than 0!");
            return;
        }
```

Figure 22: Improved code to fix tthe error

Runtime Error

Application of NumberFormatException:




```
Can only enter input while your program is running
        at java.desktop/java.awt.EventDispatchThread.pumpEvents(EventDispatchThread.java:101)
```

Figure 23: Run-time error

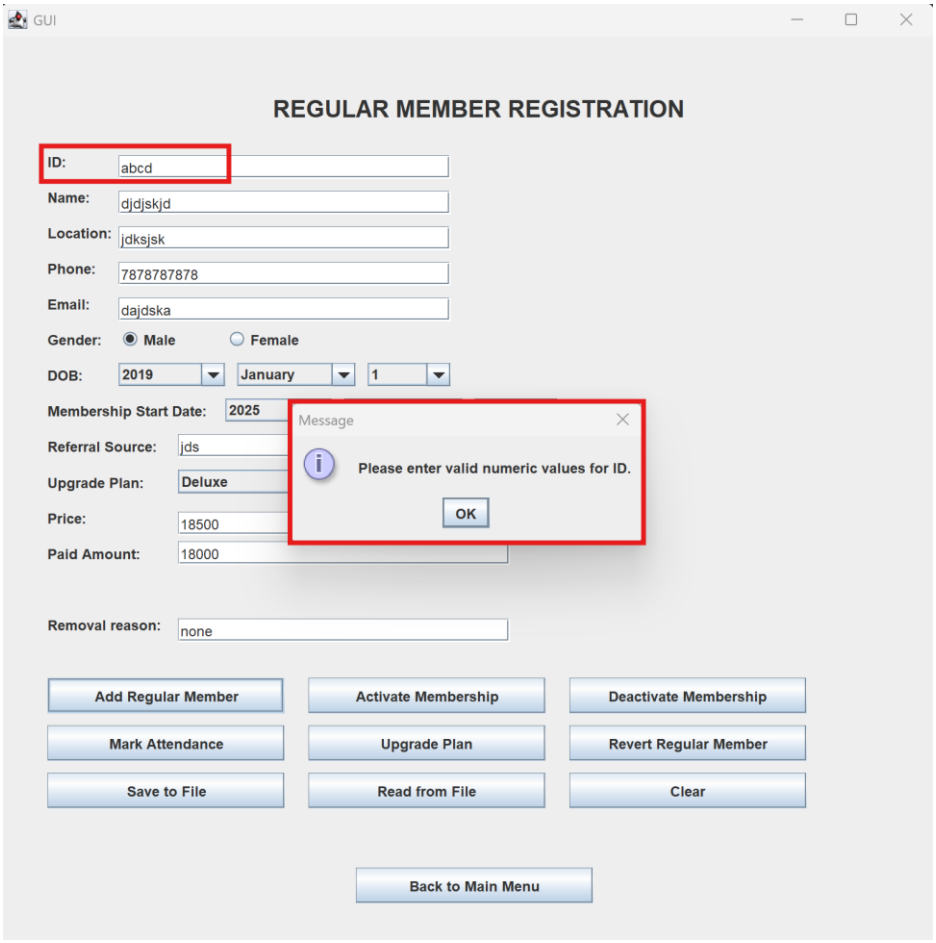


Figure 24: Fixed run-time error

```
addPremiumMemberButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try{
            int idp = Integer.parseInt(idpfield.getText());
            if (idp <= 0) {
            JOptionPane.showMessageDialog(frame, "Member ID must be greater than 0!");
            return;
        }
            clearFields();
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(frame, "Error: " + ex.getMessage());
        }
    }
});
```

Figure 25: Exception Handling

Conclusion

1. Evaluation of Work

The project effectively utilized a gym management system based on Java's object-oriented programming (OOP) principles, including inheritance, encapsulation, polymorphism, and abstraction. The abstract class GymMember and its classes RegularMember and PremiumMember were developed according to the specified requirements, each possessing all the described attributes and methods. The GUI implementation based on Swing provided an easy-to-use interface for managing members with functionalities like adding members, marking attendance, changing plans, and handling payments. The program also included file operations to save and load member details, for persistence of data.

Testing was carried out to ensure the system met all functional requirements, such as member ID checks, activating and deactivating memberships, and calculation of discounts. The code also included best practices, like exception handling and input checking, for providing robustness. Overall, the project reflected a good understanding of OOP concepts and GUI development, fulfilling the course objectives.

2. Learnings and Reflection

This project served as an ideal platform to apply theoretical OOP concepts to real-world problems. The salient learnings were:

- Inheritance and Polymorphism: Creating a class hierarchy (GymMember, RegularMember, PremiumMember) and method overriding to tailor functionality for each subclass.
- GUI Development: Building practical proficiency with Swing components, event handling, and layout management to build an interactive interface.
- File Handling: Reading and writing to files to store and load data.
- Exception Handling: Employing try-catch blocks to deal with errors in a graceful manner, such as when dealing with invalid user input or with file operations.

The project also highlighted the importance of planning and modular design, as splitting tasks into smaller methods and classes helped make it simpler to create.

3. Problems Encountered

- Abstract Classes and Methods: Understanding the concept of abstract classes and ensuring all subclasses implement abstract methods correctly can be confusing initially.
- Layout and Event Handling in GUI: Developing a good-looking and functional GUI is a question of correct planning, and aligning elements can be frustrating. Event listeners and their application might also be challenging for beginners.
- File Operations: File reading and writing data with format consistency may lead to issues like corrupted data or misparsing.
- Debugging Logical Errors: Discovery and fixing logical errors, e.g., inaccurate computation or method invocation, is labor-intensive.

4. How Obstacles Were Overcome

To address these challenges, the following methods were employed:

- Abstract Classes: Recapitated OOP basics and practiced on small projects to solidify understanding before implementing the GymMember class
- GUI Development: Used online tutorials and Swing documentation to learn about layouts and event handling. Paper prototyping of the GUI before coding enabled visualizing the layout.
- File Handling: Tested file operations line by line, ensuring data was read and written correctly before implementing them in the main program.
- Debugging: Utilized print statements and breakpoints to monitor program execution and find bugs. Peer reviews and soliciting feedback also served to catch bugs that were overlooked.

References

A.Borror, J., n.d. *Java Principles of Object Oriented Programming.* s.l.:RandD publishers.

Tsetsekas, H., n.d. *OOP Java.* 2nd ed. s.l.:Ray Yav .

Meyer, B. (1988). "*Object-Oriented Software Construction.*" Prentice Hal

McConnell, S. (2004). "*Code Complete: A Practical Handbook of Software Construction (2nd ed.)*." Microsoft Press.

Appendix

```
/**This class 'GymMember' is a abstract parent class that has two sub classes RegularMember and PremiumMember. This parent class has its
   primary attributes and methods that help carryout various functions in the aspired gym software design**/


public abstract class GymMember
{
   //attribues with prtectected access modifier
    protected int id;
   protected String name;
   protected String location;
   protected String phone;
   protected String email;
   protected String gender;
   protected String DOB;
   protected String membershipStartDate;
   protected int attendance;
   protected double loyaltyPoints;
   protected boolean activeStatus;
   //constructor
   GymMember(int id,String name, String location,String phone,
   String email, String gender, String DOB,String membershipStartDate){
    //initializing the values of some attributes
    this.attendance=0;
    this.loyaltyPoints=0.0;
    this.activeStatus=false;
    //assigning parameter values
```

```
 this.id=id;
 this.name=name;
 this.location=location;
 this.phone=phone;
 this.email=email;
 this.gender=gender;
 this.DOB=DOB;
 this.membershipStartDate=membershipStartDate;
}
//making individual accessor methods for all the given attributes
public int getId(){
    return id;
}
public String getName(){
    return name;
}
public String location(){
    return location;
}
public String phone(){
    return phone;
}
public String email(){
    return email;
}
public String gender(){
    return gender;
}
```

```
    public String DOB(){
        return DOB;
    }
    public String membershipStartDate(){
        return membershipStartDate;
    }
    public int attendance(){
        return attendance;
    }
    public double loyaltyPoints(){
        return loyaltyPoints;
    }
    public boolean activeStatus(){
        return activeStatus;
    }
    //abstract method called markAttendance to track the attendance
    public abstract void markAttendance();
    //sets membershipStatus to active
    public void activateMembership(){
        activeStatus= true;
    }
    //sets the membershipStatus to inactive if active and displays a message incase of
already inactive
    public void deactivateMembership(){
        if (activeStatus){
            activeStatus= false;}
            else{
                System.out.println("Already inactive");
            }
```

```
    }
    //resets the values of the attributes of the member
    public void resetMember(){
        activeStatus= false;
        attendance=0;
        loyaltyPoints=0.0;
    }
    //displays the inserted values
    public void display(){
        System.out.println("ID:" +id);
        System.out.println("Name:"+name);
        System.out.println("Location:"+location);
        System.out.println("Phone:"+phone);
        System.out.println("Email:"+email);
        System.out.println("Gender:"+gender);
        System.out.println("Date Of Birth:"+DOB);
        System.out.println("Membership startdate:"+membershipStartDate);
        System.out.println("Attendance:"+attendance);
        System.out.println("Loyalty points:"+loyaltyPoints);
        System.out.println("Active Status:"+ (activeStatus? "Active":"Inactive"));
    }
}
/**This class RegularMember is a child class of GymMember. It consists of various methods to manipulate various value sreferring to the
  attributes of a regular member**/

public class RegularMember extends GymMember
{
   final int attendanceLimit;
```

```
    boolean isEligibleForUpgrade;
    String removalReason;
    String referralSource;
    String plan;
    double price;
   //constructor
   RegularMember(int id, String name, String location, String phone, String email,String
gender, String DOB,String membershipStartDate,
   String referralSource){
     super(id,name,location,phone,email,gender,DOB,membershipStartDate);
     this.isEligibleForUpgrade=false;
     this.attendanceLimit=30;
     this.plan="Basic";
     this.price=6500.0;
     this.removalReason=" ";
     this.referralSource=referralSource;
   }
   //individual accessor methods
   public int getAttendanceLimit(){
       return attendanceLimit;
   }
   public boolean getIsEligibleForUpgrade(){
       return isEligibleForUpgrade;
   }
   public String getRemovalReason(){
       return removalReason;
   }
   public String getReferralSource(){
       return referralSource;
```

```
}
public String getPlan(){
    return plan;
}
public double getPrice(){
    return price;
}
//to display the price of chosen plan
public double getPlanPrice(String plan){
    switch(plan.toLowerCase()){
        case "basic":
            return 6500.0;
         case "standard":
             return 12500.0;
         case "deluxe":
             return 18500.0;
         default:
             return -1;
    }
}
public String upgradePlan(String newPlan){
    if(attendance>=attendanceLimit){
        isEligibleForUpgrade= true;
    }
    if(!isEligibleForUpgrade){
        return "Member is not eligible for an upgrade";
    }
    if(plan.equalsIgnoreCase(newPlan)){
```

```
        return "You are already subscribed to this plan";
    }
    double newPrice = getPlanPrice(newPlan);
    if(newPrice ==-1){
        return "invalid plan selected";
    }
    this.plan=newPlan;
    this.price=newPrice;
    return "Plan successfully upgraded to" +newPlan+ "for" +newPrice;
}
public void revertRegularMember(String removalReason){
    resetMember();
    this.isEligibleForUpgrade =false;
    this.plan="Basic";
    this.price=6500.0;
    this.removalReason=removalReason;
}
@Override
public void markAttendance(){
    if(attendance<attendanceLimit){
        attendance++;
        loyaltyPoints+=5;}
        else{
            System.out.println("Attendance limit reached for regular member.");
        }
    }
  @Override
  public void display(){
```

```
        super.display();
        System.out.println("Plan:"+plan);
        System.out.println("Price:"+price);
        if(!removalReason.isEmpty()){
            System.out.println("Removal reason:"+removalReason);
        }
    }
  }

/**This class PremiumMember is a child class of GymMember. This class consists of
various methods that help store, update and manipulate
   values referring to the attributes of Premium member**/
public class PremiumMember extends GymMember
{
     final double premiumCharge =50000;
     String personalTrainer;
     boolean isFullPayment=false;
     double paidAmount=0;
     double discountAmount=0;
    //constructor
    PremiumMember(int id, String name,String location, String phone, String email,
    String gender, String DOB, String membershipStartDate, String personalTrainer){
        super(id, name, location, phone, email, gender, DOB, membershipStartDate);
        this.personalTrainer= personalTrainer;
    }
    // Accessor methods
    public String getPersonalTrainer() {
        return personalTrainer;
    }
```

```
public boolean getIsFullPayment() {
    return isFullPayment;
}

public double getPaidAmount() {
    return paidAmount;
}

public double getDiscountAmount() {
    return discountAmount;
}
public String payDueAmount(double payment){

    if (payment >= 50000) {
    // Apply 10% discount
    double discount = payment * 0.10;
    double finalAmount = payment - discount;

    // Update the member's payment record
    this.paidAmount = finalAmount;
    this.discountAmount = discount;
    this.isFullPayment = true;
    return "Payment successful!\nOriginal Amount: " + payment +
            "\nDiscount (10%): " + discount +
            "\nFinal Amount Paid: " + finalAmount;

} else {
```

```
        // Partial payment without discount
        this.paidAmount = payment;
        double remaining = 50000 - payment;

        return "Partial payment of " + payment + " received.\nRemaining balance: " +
remaining;

    }

    }
    public void calculateDiscount() {
        if (isFullPayment) {
            discountAmount = premiumCharge * 0.10;
            System.out.println("Discount given: " + discountAmount);
        } else {
            System.out.println("No discount available as payment is not complete.");
        }
    }
    public void revertPremiumMember(){
        super.resetMember();
        this.personalTrainer=" ";
        this.isFullPayment= false;
        this.paidAmount=0;
        this.discountAmount=0;
    }
    @Override
    public void markAttendance(){
      }
    @Override
```

```
    public void display(){
        super.display();
        System.out.println("Personal trainer:"+personalTrainer);
        System.out.println("Paid amount:"+paidAmount);
        System.out.println("Full payment:"+isFullPayment);
        double remainingAmount=premiumCharge-paidAmount;
        System.out.println("remaining amount:"+remainingAmount);
        if(isFullPayment){
            System.out.println("discounf amount:"+discountAmount);
        }
    }
}    import javax.swing.JFrame;
    import java.util.ArrayList;
    import javax.swing.JButton;
    import javax.swing.JLabel;
    import javax.swing.JTextField;
    import javax.swing.JComboBox;
    import javax.swing.JRadioButton;
    import javax.swing.ButtonGroup;
    import javax.swing.JPanel;
    import java.awt.event.ActionListener;
    import java.awt.event.ActionEvent;
    import javax.swing.JOptionPane;
    import javax.swing.JTextArea;
    import javax.swing.JScrollPane;
    import java.awt.Font;
    import java.awt.Color;
    import java.io.FileWriter;
```

```
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
import java.awt.BorderLayout;


public class GymGUI {
    JFrame frame;
    JPanel mainPanel, rPanel, pPanel, displayPanel;
    JLabel title, regularLabel, id, name, location, phone, email, gender, dob,
    membershipStDate, referralSource, upgradePlan,removalreason, price, paidAmt,
discount,

premiumLabel,idp,namep,locationp,phonep,emailp,genderp,dobp,membershipStDatep,r
emovalreasonp,
    pricep,pricevaluep,paidAmtp,discountp,personalTrainerp;
    // Initialize the text fields here to prevent null pointer exceptions
    JTextField idfield = new JTextField();
    JTextField namefield = new JTextField();
    JTextField locationfield = new JTextField();
    JTextField phonefield = new JTextField();
    JTextField emailfield = new JTextField();
    JTextField referralSourcefield = new JTextField();
    JTextField paidAmtfield = new JTextField();
    JTextField pricefield=new JTextField();
    JTextField removalreasonfield=new JTextField();
    JTextField idpfield = new JTextField();
    JTextField namepfield = new JTextField();
    JTextField locationpfield = new JTextField();
```

```
    JTextField phonepfield = new JTextField();
    JTextField emailpfield = new JTextField();
    JTextField paidAmtpfield = new JTextField();
    JTextField removalreasonpfield=new JTextField();
    JTextField personalTrainerpfield=new JTextField();
    JRadioButton maleRadioButton, femaleRadioButton,
maleRadioButtonp,femaleRadioButtonp;
    JComboBox<String> dobYear, dobMonth, dobDay, plan, startYear, startMonth,
startDay,
    dobYearp,dobMonthp,dobDayp,startYearp,startMonthp,startDayp;
    ArrayList<String> regularMembers;
    ArrayList<String> premiumMembers;
    ArrayList<GymMember> membersList;

    public static void main (String[]args){
        new GymGUI();
    }

    private void clearfields(){
        idfield.setText("");
        namefield.setText("");
        locationfield.setText("");
        phonefield.setText("");
        emailfield.setText("");
        referralSourcefield.setText("");
        paidAmtfield.setText("");
        dobYear.setSelectedIndex(0);
        dobMonth.setSelectedIndex(0);
        dobDay.setSelectedIndex(0);
```

```
    startYear.setSelectedIndex(0);
    startMonth.setSelectedIndex(0);
    startDay.setSelectedIndex(0);

    idpfield.setText("");
    namepfield.setText("");
    locationpfield.setText("");
    phonepfield.setText("");
    emailpfield.setText("");
    paidAmtpfield.setText("");
    removalreasonpfield.setText("");
    personalTrainerpfield.setText("");
    dobYearp.setSelectedIndex(0);
    dobMonthp.setSelectedIndex(0);
    dobDayp.setSelectedIndex(0);
    startYearp.setSelectedIndex(0);
    startMonthp.setSelectedIndex(0);
    startDayp.setSelectedIndex(0);
}

public GymGUI() {
    regularMembers = new ArrayList<>();
    premiumMembers = new ArrayList<>();
    membersList = new ArrayList<>();

    frame = new JFrame("GUI");
    frame.setSize(800, 800);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
frame.setLayout(null);

mainPanel = new JPanel();
mainPanel.setBounds(0, 0, 800, 800);
mainPanel.setLayout(null);
mainPanel.setVisible(true);

rPanel = new JPanel();
rPanel.setBounds(0, 0, 800, 800);
rPanel.setLayout(null);
rPanel.setVisible(false);

pPanel = new JPanel();
pPanel.setBounds(0, 0, 800, 800);
pPanel.setLayout(null);
pPanel.setVisible(false);

displayPanel = new JPanel();
displayPanel.setBounds(0, 0, 800, 800);
displayPanel.setLayout(null);
displayPanel.setVisible(false);

frame.add(mainPanel);
frame.add(rPanel);
frame.add(pPanel);
frame.add(displayPanel);

setupMainPanel();
```

```
    setuprPanel();

    setuppPanel();

    setupdisplayPanel();

    frame.setVisible(true);
}

private void setupMainPanel() {
    mainPanel.setBackground(new java.awt.Color(52,70,70));

    title = new JLabel("GYM MEMBERSHIP SYSTEM");
    title.setForeground(Color.WHITE);
    title.setFont(new Font("Montserrat", Font.BOLD, 30));

    JButton addRegularButton = new JButton("Add a Regular Member");
    JButton addPremiumButton = new JButton("Add a Premium Member");
    JButton showMembersButton = new JButton("Show Members");

    title.setBounds(200, 80, 500, 80);
    mainPanel.add(title);
    addRegularButton.setBounds(300, 200, 200, 40);
    mainPanel.add(addRegularButton);
    addPremiumButton.setBounds(300, 300, 200, 40);
    mainPanel.add(addPremiumButton);
    showMembersButton.setBounds(300, 400, 200, 40);
    mainPanel.add(showMembersButton);
```

```
    addRegularButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                mainPanel.setVisible(false);
                rPanel.setVisible(true);
                pPanel.setVisible(false);
                displayPanel.setVisible(false);
            }
        });
    addPremiumButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                mainPanel.setVisible(false);
                rPanel.setVisible(false);
                pPanel.setVisible(true);
                displayPanel.setVisible(false);
            }
        });
    showMembersButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                mainPanel.setVisible(false);
                rPanel.setVisible(false);
                pPanel.setVisible(false);
                displayPanel.setVisible(true);
            }
        });
}

private void setuprPanel() {
```

```
regularLabel = new JLabel("REGULAR MEMBER REGISTRATION");
regularLabel.setFont(new Font("Montserrat", Font.BOLD, 20));
regularLabel.setBounds(230, 20, 400, 80);
rPanel.add(regularLabel);

id = new JLabel("ID:");
id.setBounds(40, 80, 100, 50);
rPanel.add(id);
idfield.setBounds(100, 100, 280, 20);
rPanel.add(idfield);

name = new JLabel("Name:");
name.setBounds(40, 110, 100, 50);
rPanel.add(name);
namefield.setBounds(100, 130, 280, 20);
rPanel.add(namefield);

location = new JLabel("Location:");
location.setBounds(40, 140, 100, 50);
rPanel.add(location);
locationfield.setBounds(100, 160, 280, 20);
rPanel.add(locationfield);

phone = new JLabel("Phone:");
phone.setBounds(40, 170, 100, 50);
rPanel.add(phone);
phonefield.setBounds(100, 190, 280, 20);
rPanel.add(phonefield);
```

```
email = new JLabel("Email:");
email.setBounds(40, 200, 100, 50);
rPanel.add(email);
emailfield.setBounds(100, 220, 280, 20);
rPanel.add(emailfield);

gender = new JLabel("Gender:");
gender.setBounds(40, 230, 100, 50);
rPanel.add(gender);

// Initialize radio buttons
maleRadioButton = new JRadioButton("Male");
femaleRadioButton = new JRadioButton("Female");
maleRadioButton.setBounds(100, 245, 80, 20);
femaleRadioButton.setBounds(190, 245, 80, 20);

// Create button group to ensure only one radio button can be selected
ButtonGroup genderGroup = new ButtonGroup();
genderGroup.add(maleRadioButton);
genderGroup.add(femaleRadioButton);

rPanel.add(maleRadioButton);
rPanel.add(femaleRadioButton);

dob = new JLabel("DOB:");
dob.setBounds(40, 260, 100, 50);
rPanel.add(dob);
```

```
// Initialize date components
String[] years = getYears();
String[] months = {"January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"};
String[] days = getDays();

dobYear = new JComboBox<>(years);
dobMonth = new JComboBox<>(months);
dobDay = new JComboBox<>(days);

dobYear.setBounds(100, 275, 90, 20);
dobMonth.setBounds(200, 275, 100, 20);
dobDay.setBounds(310, 275, 70, 20);

rPanel.add(dobYear);
rPanel.add(dobMonth);
rPanel.add(dobDay);

membershipStDate = new JLabel("Membership Start Date:");
membershipStDate.setBounds(40, 290, 150, 50);
rPanel.add(membershipStDate);

startYear = new JComboBox<>(years);
startMonth = new JComboBox<>(months);
startDay = new JComboBox<>(days);

startYear.setBounds(190, 305, 90, 20);
```

```
startMonth.setBounds(290, 305, 100, 20);
startDay.setBounds(400, 305, 70, 20);

rPanel.add(startYear);
rPanel.add(startMonth);
rPanel.add(startDay);

referralSource = new JLabel("Referral Source:");
referralSource.setBounds(40, 320, 100, 50);
rPanel.add(referralSource);
referralSourcefield.setBounds(150, 335, 280, 20);
rPanel.add(referralSourcefield);

upgradePlan = new JLabel("Upgrade Plan:");
upgradePlan.setBounds(40, 350, 100, 50);
rPanel.add(upgradePlan);

String[] plans = {"Basic", "Deluxe", "Premium"};
plan = new JComboBox<>(plans);
plan.setBounds(150, 365, 150, 20);
rPanel.add(plan);

price = new JLabel("Price:");
price.setBounds(40, 380, 100, 50);
rPanel.add(price);

pricefield.setBounds(150,400,200,20);
rPanel.add(pricefield);
```

```
paidAmt = new JLabel("Paid Amount:");
paidAmt.setBounds(40, 410, 100, 50);
rPanel.add(paidAmt);
paidAmtfield.setBounds(150, 425, 280, 20);
rPanel.add(paidAmtfield);

JLabel removalreason=new JLabel("Removal reason:");
removalreason.setBounds(40,470,100,50);
rPanel.add(removalreason);
removalreasonfield.setBounds(150,490,280,20);
rPanel.add(removalreasonfield);

JButton addRegularMemberButton = new JButton("Add Regular Member");
addRegularMemberButton.setBounds(40, 540, 200, 30);
rPanel.add(addRegularMemberButton);

JButton activateMembershipButton = new JButton("Activate Membership");
activateMembershipButton.setBounds(260, 540, 200, 30);
rPanel.add(activateMembershipButton);

JButton deactivateMembershipButton = new JButton("Deactivate Membership");
deactivateMembershipButton.setBounds(480, 540, 200, 30);
rPanel.add(deactivateMembershipButton);

JButton markAttendanceButton = new JButton("Mark Attendance");
markAttendanceButton.setBounds(40, 580, 200, 30);
rPanel.add(markAttendanceButton);
```

```
JButton upgradePlanButton = new JButton("Upgrade Plan");
upgradePlanButton.setBounds(260, 580, 200, 30);
rPanel.add(upgradePlanButton);

JButton revertRegularMemberButton = new JButton("Revert Regular Member");
revertRegularMemberButton.setBounds(480, 580, 200, 30);
rPanel.add(revertRegularMemberButton);

JButton clearButton = new JButton("Clear");
clearButton.setBounds(480, 620, 200, 30);
rPanel.add(clearButton);

JButton saveToFileButton = new JButton("Save to File");
saveToFileButton.setBounds(40, 620, 200, 30);
rPanel.add(saveToFileButton);

JButton readFromFileButton = new JButton("Read from File");
readFromFileButton.setBounds(260, 620, 200, 30);
rPanel.add(readFromFileButton);



// Add a back button to return to main panel
JButton backButton = new JButton("Back to Main Menu");
backButton.setBounds(300, 700, 200, 30);
rPanel.add(backButton);

backButton.addActionListener(new ActionListener() {
```

```
            public void actionPerformed(ActionEvent e) {
                rPanel.setVisible(false);
                mainPanel.setVisible(true);
            }
        });

        plan.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String selectedPlan = (String) plan.getSelectedItem();
                if (selectedPlan.equals("Basic")) {
                    pricefield.setText("6500");
                } else if (selectedPlan.equals("Standard")) {
                    pricefield.setText("12500");
                } else if (selectedPlan.equals("Deluxe")) {
                    pricefield.setText("18500");
                }
            }
        });

        // Add action listeners for buttons
        addRegularMemberButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                int id = Integer.parseInt(idfield.getText());

                if (id <= 0) {
                    JOptionPane.showMessageDialog(frame, "Member ID must be
greater than 0!");
                    return;
```

```
                }
                    String name = namefield.getText();
                    String location = locationfield.getText();
                    String phone = phonefield.getText();
                    String email = emailfield.getText();
                    if (phone.length() != 10 || !phone.matches("\\d{10}")) {
                    JOptionPane.showMessageDialog(frame, "Phone number must be
exactly 10 digits!");
                    return;
                }
                    String gender = maleRadioButton.isSelected() ? "Male" : "Female";
                    //String dob = dobYear.getText();
                    //String startDate = membershipStartField.getText();
                    String referral = referralSourcefield.getText();

                    // Check for duplicate ID
                    for (GymMember member : (membersList)){
                       if (member.getId() == id) {
                           JOptionPane.showMessageDialog(frame, "Member ID already
exists!");
                           return;
                       }
                    }

                    // Get the date from the dropdowns
                    String dobString = dobYear.getSelectedItem() + "-" +
dobMonth.getSelectedItem() + "-" + dobDay.getSelectedItem();
                    String startDateString = startYear.getSelectedItem() + "-" +
startMonth.getSelectedItem() + "-" + startDay.getSelectedItem();
                    // Create new RegularMember and add to list
```

```
                    RegularMember newMember = new RegularMember(id, name,
location, phone, email, gender, dobString,startDateString, referral);
                    membersList.add(newMember);


                    JOptionPane.showMessageDialog(frame, "Regular Member added
successfully!");
                    clearfields();
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(frame, "Please enter valid numeric
values for ID.");
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(frame, "Error: " +
ex.getMessage());
                }
            }
        });


        activateMembershipButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int id = Integer.parseInt(idfield.getText());
                    boolean found = false;


                    for (GymMember member : membersList) {
                        if (member.getId() == id) {
                            member.activateMembership();
                            JOptionPane.showMessageDialog(frame, "Membership activated
successfully!");
                            found = true;
                            break;
```

```
                    }
                }

                if (!found) {
                    JOptionPane.showMessageDialog(frame, "Member ID not found!");
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
            }
        }
    });

    deactivateMembershipButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            try {
                int id = Integer.parseInt(idfield.getText());
                boolean found = false;

                for (GymMember member : membersList) {
                    if (member.getId() == id) {
                        member.deactivateMembership();
                        JOptionPane.showMessageDialog(frame, "Membership deactivated successfully!");
                        found = true;
                        break;
                    }
                }
```

```
                    if (!found) {
                        JOptionPane.showMessageDialog(frame, "Member ID not found!");
                    }
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
                }
            }
        });

        markAttendanceButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int id = Integer.parseInt(idfield.getText());
                    boolean found = false;

                    for (GymMember member : membersList) {
                        if (member.getId() == id) {
                            if (member.activeStatus()) {
                                member.markAttendance();
                                JOptionPane.showMessageDialog(frame, "Attendance marked successfully!");
                            } else {
                                JOptionPane.showMessageDialog(frame, "Member is not active. Cannot mark attendance.");
                            }
                            found = true;
                            break;
                        }
```

```
                    }

                    if (!found) {
                        JOptionPane.showMessageDialog(frame, "Member ID not found!");
                    }
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
                }
            }
        });

        upgradePlanButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int id = Integer.parseInt(idfield.getText());
                    String selectedPlan = (String) plan.getSelectedItem();
                    boolean found = false;

                    for (GymMember member : membersList) {
                        if (member.getId() == id) {
                            if (member instanceof RegularMember) {
                                if (member.activeStatus()) {
                                    RegularMember regMember = (RegularMember) member;
                                    String result = regMember.upgradePlan(selectedPlan);
                                    JOptionPane.showMessageDialog(frame, result);
                                } else {
                                    JOptionPane.showMessageDialog(frame, "Member is not active. Cannot upgrade plan.");
```

```
                    }
                } else {
                    JOptionPane.showMessageDialog(frame, "This member is not a Regular Member!");
                }
                found = true;
                break;
            }
        }

        if (!found) {
            JOptionPane.showMessageDialog(frame, "Member ID not found!");
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
    }
  }
});

revertRegularMemberButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            int id = Integer.parseInt(idfield.getText());
            String reason = removalreasonfield.getText();
            boolean found = false;

            for (GymMember member : membersList) {
                if (member.getId() == id) {
```

```
                        if (member instanceof RegularMember) {
                            RegularMember regularMember = (RegularMember) member;
                            regularMember.revertRegularMember(reason);
                            JOptionPane.showMessageDialog(frame, "Regular Member
reverted successfully!");
                        } else {
                            JOptionPane.showMessageDialog(frame, "This member is not
a Regular Member!");
                        }
                        found = true;
                        break;
                    }
                }

                if (!found) {
                    JOptionPane.showMessageDialog(frame, "Member ID not found!");
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(frame, "Please enter a valid
numeric Member ID.");
            }
        }
    });

    clearButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            clearfields();
        }
    });
```

```
    saveToFileButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                saveToFile();
            }
        });

    readFromFileButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                readFromFile();
            }
        });
}

// Helper method to generate years for dropdown
private String[] getYears() {
    String[] years = new String[100];
    int currentYear = 2025;
    for (int i = 0; i < years.length; i++) {
        years[i] = String.valueOf(currentYear - i);
    }
    return years;
}

// Helper method to generate days for dropdown
private String[] getDays() {
    String[] days = new String[31];
    for (int i = 0; i < days.length; i++) {
        days[i] = String.valueOf(i + 1);
```

```
    }
    return days;
}

private void setuppPanel() {
    premiumLabel = new JLabel("PREMIUM MEMBER REGISTRATION");
    premiumLabel.setFont(new Font("Montserrat", Font.BOLD, 20));
    premiumLabel.setBounds(230, 20, 400, 80);
    pPanel.add(premiumLabel);

    idp = new JLabel("ID:");
    idp.setBounds(40, 80, 100, 50);
    pPanel.add(idp);
    idpfield.setBounds(100, 100, 280, 20);
    pPanel.add(idpfield);

    namep = new JLabel("Name:");
    namep.setBounds(40, 110, 100, 50);
    pPanel.add(namep);
    namepfield.setBounds(100, 130, 280, 20);
    pPanel.add(namepfield);

    locationp = new JLabel("Location:");
    locationp.setBounds(40, 140, 100, 50);
    pPanel.add(locationp);
    locationpfield.setBounds(100, 160, 280, 20);
    pPanel.add(locationpfield);
```

```
phonep = new JLabel("Phone:");
phonep.setBounds(40, 170, 100, 50);
pPanel.add(phonep);
phonepfield.setBounds(100, 190, 280, 20);
pPanel.add(phonepfield);

emailp = new JLabel("Email:");
emailp.setBounds(40, 200, 100, 50);
pPanel.add(emailp);
emailpfield.setBounds(100, 220, 280, 20);
pPanel.add(emailpfield);

genderp = new JLabel("Gender:");
genderp.setBounds(40, 230, 100, 50);
pPanel.add(genderp);

// Initialize radio buttons
maleRadioButtonp = new JRadioButton("Male");
femaleRadioButtonp = new JRadioButton("Female");
maleRadioButtonp.setBounds(100, 245, 80, 20);
femaleRadioButtonp.setBounds(190, 245, 80, 20);

// Create button group to ensure only one radio button can be selected
ButtonGroup genderpGroup = new ButtonGroup();
genderpGroup.add(maleRadioButtonp);
genderpGroup.add(femaleRadioButtonp);

pPanel.add(maleRadioButtonp);
```

```
pPanel.add(femaleRadioButtonp);

dobp = new JLabel("DOB:");
dobp.setBounds(40, 260, 100, 50);
pPanel.add(dobp);

// Initialize date components
String[] years = getYearsp();
String[] months = {"January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"};
String[] days = getDaysp();

dobYearp = new JComboBox<>(years);
dobMonthp = new JComboBox<>(months);
dobDayp = new JComboBox<>(days);

dobYearp.setBounds(100, 275, 90, 20);
dobMonthp.setBounds(200, 275, 100, 20);
dobDayp.setBounds(310, 275, 70, 20);

pPanel.add(dobYearp);
pPanel.add(dobMonthp);
pPanel.add(dobDayp);

membershipStDatep = new JLabel("Membership Start Date:");
membershipStDatep.setBounds(40, 290, 150, 50);
pPanel.add(membershipStDatep);
```

```
startYearp = new JComboBox<>(years);
startMonthp = new JComboBox<>(months);
startDayp = new JComboBox<>(days);

startYearp.setBounds(190, 305, 90, 20);
startMonthp.setBounds(290, 305, 100, 20);
startDayp.setBounds(400, 305, 70, 20);

pPanel.add(startYearp);
pPanel.add(startMonthp);
pPanel.add(startDayp);

personalTrainerp = new JLabel("Personal Trainer:");
personalTrainerp.setBounds(40, 320, 100, 50);
pPanel.add(personalTrainerp);
personalTrainerpfield.setBounds(150, 335, 280, 20);
pPanel.add(personalTrainerpfield);

pricep = new JLabel("Price:");
pricep.setBounds(40, 380, 100, 50);
pPanel.add(pricep);

pricevaluep=new JLabel("50,000");
pricevaluep.setBounds(150,400,200,20);
pPanel.add(pricevaluep);

paidAmtp = new JLabel("Paid Amount:");
paidAmtp.setBounds(40, 410, 100, 50);
```

```
        pPanel.add(paidAmtp);
        paidAmtpfield.setBounds(150, 425, 280, 20);
        pPanel.add(paidAmtpfield);

        discountp = new JLabel("Discount:");
        discountp.setBounds(40, 440, 100, 50);
        pPanel.add(discountp);

        JLabel removalreasonp=new JLabel("Removal reason:");
        removalreasonp.setBounds(40,470,100,50);
        pPanel.add(removalreasonp);
        removalreasonpfield.setBounds(150,490,280,20);
        pPanel.add(removalreasonpfield);

        JButton addPremiumMemberButton = new JButton("Add Premium Member");
        addPremiumMemberButton.setBounds(40, 540, 200, 30);
        pPanel.add(addPremiumMemberButton);

        JButton activateMembershipButtonp = new JButton("Activate Membership");
        activateMembershipButtonp.setBounds(260, 540, 200, 30);
        pPanel.add(activateMembershipButtonp);

        JButton deactivateMembershipButtonp = new JButton("Deactivate
Membership");
        deactivateMembershipButtonp.setBounds(480, 540, 200, 30);
        pPanel.add(deactivateMembershipButtonp);

        JButton markAttendanceButtonp = new JButton("Mark Attendance");
        markAttendanceButtonp.setBounds(40, 580, 200, 30);
```

```
        pPanel.add(markAttendanceButtonp);

        JButton revertPremiumMemberButton = new JButton("Revert Premium
Member");
        revertPremiumMemberButton.setBounds(480, 580, 200, 30);
        pPanel.add(revertPremiumMemberButton);

        JButton clearButtonp = new JButton("Clear");
        clearButtonp.setBounds(260, 580, 200, 30);
        pPanel.add(clearButtonp);

        JButton saveToFileButtonp = new JButton("Save to File");
        saveToFileButtonp.setBounds(40, 620, 200, 30);
        pPanel.add(saveToFileButtonp);

        // Add read from file button
        JButton readFromFileButtonp = new JButton("Read from File");
        readFromFileButtonp.setBounds(260, 620, 200, 30);
        pPanel.add(readFromFileButtonp);

        JButton calculateDiscountButton = new JButton("Calculate Discount");
        calculateDiscountButton.setBounds(160, 450, 200, 30);
        pPanel.add(calculateDiscountButton);

        JLabel discountAmountLabel = new JLabel("");
        discountAmountLabel.setBounds(150, 450, 200, 20);
        pPanel.add(discountAmountLabel);

        JButton payDueAmountButton = new JButton("Pay Due Amount");
```

```
        payDueAmountButton.setBounds(480, 620, 200, 20);
        pPanel.add(payDueAmountButton);

        // Add a back button to return to main panel
        JButton backButtonp = new JButton("Back to Main Menu");
        backButtonp.setBounds(300, 700, 200, 30);
        pPanel.add(backButtonp);

        backButtonp.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                pPanel.setVisible(false);
                mainPanel.setVisible(true);
            }
        });
        addPremiumMemberButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try{
                    int idp = Integer.parseInt(idpfield.getText());
                    if (idp <= 0) {
                    JOptionPane.showMessageDialog(frame, "Member ID must be
greater than 0!");
                    return;
                }
                    String namep = namepfield.getText();
                    String locationp = locationpfield.getText();
                    String phonep = phonepfield.getText();
                    String emailp = emailpfield.getText();
                    if (phonep.length() != 10 || !phonep.matches("\\d{10}")) {
```

```
                    JOptionPane.showMessageDialog(frame, "Phone number must be
exactly 10 digits!");
                    return;
                }
                    String genderp = maleRadioButtonp.isSelected() ? "Male" : "Female";
                    String personalTrainerp=personalTrainerpfield.getText();
                    //String dob = dobYear.getText();
                    //String startDate = membershipStartField.getText();

                    // Check for duplicate ID
                    for (GymMember member : (membersList)){
                       if (member.getId() == idp) {
                           JOptionPane.showMessageDialog(frame, "Member ID already
exists!");
                           return;
                       }
                    }

                    // Get the date from the dropdowns
                    String dobStringp = dobYearp.getSelectedItem() + "-" +
dobMonthp.getSelectedItem() + "-" + dobDayp.getSelectedItem();
                    String startDateStringp = startYearp.getSelectedItem() + "-" +
startMonthp.getSelectedItem() + "-" + startDayp.getSelectedItem();
                    // Create new RegularMember and add to list
                    PremiumMember newMember = new PremiumMember(idp, namep,
locationp, phonep, emailp, genderp, dobStringp, startDateStringp,personalTrainerp);
                    membersList.add(newMember);

                    JOptionPane.showMessageDialog(frame, "Premium Member added
successfully!");
```

```
                    clearfields();
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(frame, "Error: " + ex.getMessage());
                }
            }
        });

        activateMembershipButtonp.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int idp = Integer.parseInt(idpfield.getText());
                    boolean found = false;

                    for (GymMember member : membersList) {
                        if (member.getId() == idp) {
                            member.activateMembership();
                            JOptionPane.showMessageDialog(frame, "Membership activated successfully!");
                            found = true;
                            break;
                        }
                    }

                    if (!found) {
                        JOptionPane.showMessageDialog(frame, "Member ID not found!");
```

```
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
            }
        }
    });

    deactivateMembershipButtonp.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            try {
                int idp = Integer.parseInt(idpfield.getText());
                boolean found = false;

                for (GymMember member : membersList) {
                    if (member.getId() == idp) {
                        member.deactivateMembership();
                        JOptionPane.showMessageDialog(frame, "Membership deactivated successfully!");
                        found = true;
                        break;
                    }
                }

                if (!found) {
                    JOptionPane.showMessageDialog(frame, "Member ID not found!");
                }
            } catch (NumberFormatException ex) {
```

```
                    JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
                }
            }
        });

        markAttendanceButtonp.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int idp = Integer.parseInt(idpfield.getText());
                    boolean found = false;

                    for (GymMember member : membersList) {
                        if (member.getId() == idp) {
                            if (member.activeStatus()) {
                                member.markAttendance();
                                JOptionPane.showMessageDialog(frame, "Attendance marked successfully!");
                            } else {
                                JOptionPane.showMessageDialog(frame, "Member is not active. Cannot mark attendance.");
                            }
                            found = true;
                            break;
                        }
                    }

                    if (!found) {
                        JOptionPane.showMessageDialog(frame, "Member ID not found!");
```

```
                    }
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(frame, "Please enter valid numeric values for ID.");
                }
            }
        });



        calculateDiscountButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int id = Integer.parseInt(idpfield.getText());
                    boolean found = false;

                    for (GymMember member : membersList) {
                        if (member.getId() == id) {
                            if (member instanceof PremiumMember) {
                                PremiumMember premiumMember = (PremiumMember) member;
                                double standardPrice = 50000;
                                double discount = standardPrice * 0.10;
                                double discountedPrice = standardPrice - discount;

                                JOptionPane.showMessageDialog(frame,
                                "Discount Calculation:\n" +
                                "Original Price: 50,000\n" +
                                "Discount (10%): " + discount + "\n" +
```

```
                        "Price after discount: " + discountedPrice);
                    } else {
                        JOptionPane.showMessageDialog(frame, "This member is not
a Premium Member!");
                    }
                    found = true;
                    break;
                }
            }

            if (!found) {
                JOptionPane.showMessageDialog(frame, "Member ID not found!");
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(frame, "Please enter a valid
numeric Member ID.");
        }
    }
});

payDueAmountButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            int idp = Integer.parseInt(idpfield.getText());
            double payment = Double.parseDouble(paidAmtpfield.getText());
            boolean found = false;

            for (GymMember member : membersList) {
                //here
```

```
                    if (member.getId() == idp) {
                        if (member instanceof PremiumMember) {
                            PremiumMember premiumMember = (PremiumMember)
member;
                            String result = premiumMember.payDueAmount(payment);
                            JOptionPane.showMessageDialog(frame, result);
                             if (premiumMember.getIsFullPayment()) {
                    discountp.setText("Discount: " +
premiumMember.getDiscountAmount());
                } else {
                    discountp.setText("Discount: 0.0");
                }
                        } else {
                            JOptionPane.showMessageDialog(frame, "This member is not
a Premium Member!");
                        }
                        found = true;
                        break;
                    }
                }

                if (!found) {
                    JOptionPane.showMessageDialog(frame, "Member ID not found!");
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(frame, "Please enter valid numeric
values for Member ID and Payment.");
            }
        }
    });
```

```
        revertPremiumMemberButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int id = Integer.parseInt(idpfield.getText());
                    String reason = removalreasonpfield.getText();
                    boolean found = false;

                    for (GymMember member : membersList) {
                        if (member.getId() == id) {
                            if (member instanceof PremiumMember) {
                                PremiumMember premiumMember = (PremiumMember) member;
                                premiumMember.revertPremiumMember();
                                JOptionPane.showMessageDialog(frame, "Premium Member reverted successfully!");
                            } else {
                                JOptionPane.showMessageDialog(frame, "This member is not a Premium Member!");
                            }
                            found = true;
                            break;
                        }
                    }

                    if (!found) {
                        JOptionPane.showMessageDialog(frame, "Member ID not found!");
                    }
                } catch (NumberFormatException ex) {
```

```
                    JOptionPane.showMessageDialog(frame, "Please enter a valid
numeric Member ID.");
                }
            }
        });

        clearButtonp.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                clearfields();
            }
        });

        saveToFileButtonp.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                saveToFile();
            }
        });

        readFromFileButtonp.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                readFromFile();
            }
        });
    }

    // Helper method to generate years for dropdown
    private String[] getYearsp() {
        String[] years = new String[100];
        int currentYear = 2025;
```

```
    for (int i = 0; i < years.length; i++) {
        years[i] = String.valueOf(currentYear - i);
    }
    return years;
}

// Helper method to generate days for dropdown
private String[] getDaysp() {
    String[] days = new String[31];
    for (int i = 0; i < days.length; i++) {
        days[i] = String.valueOf(i + 1);
    }
    return days;
}

private void setupdisplayPanel() {
    // Title
    JLabel title = new JLabel("All Gym Members");
    title.setBounds(300, 20, 200, 30);
    displayPanel.add(title);

    // Text area to display members
    JTextArea membersArea = new JTextArea();
    membersArea.setEditable(false);
    JScrollPane scroll = new JScrollPane(membersArea);
    scroll.setBounds(50, 60, 700, 600);
    displayPanel.add(scroll);
```

```
// Refresh button
JButton refreshBtn = new JButton("Refresh");
refreshBtn.setBounds(300, 680, 100, 30);
displayPanel.add(refreshBtn);

JButton saveToFileButtond = new JButton("Save to File");
saveToFileButtond.setBounds(200, 680, 100, 30);
displayPanel.add(saveToFileButtond);

// Add read from file button
JButton readFromFileButtond = new JButton("Read from File");
readFromFileButtond.setBounds(550, 680, 100, 30);
displayPanel.add(readFromFileButtond);

// Back button
JButton backBtn = new JButton("Back");
backBtn.setBounds(420, 680, 100, 30);
displayPanel.add(backBtn);

// Display members when panel opens
showAllMembers(membersArea);

// Add action listeners for the new buttons
saveToFileButtond.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            saveToFile();
        }
    });
```

```
        readFromFileButtond.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                readFromFile();
            }
        });
    }

    private void showAllMembers(JTextArea area) {
        StringBuilder sb = new StringBuilder();

        if (membersList.isEmpty()) {
            sb.append("No members registered yet.\n");
        } else {
            sb.append("Regular Members\n");
            for (GymMember member : membersList) {
                if (member instanceof RegularMember) {
                    RegularMember rm = (RegularMember) member;
                    sb.append("ID: ").append(rm.getId()).append("\n");
                    sb.append("Name: ").append(rm.getName()).append("\n");
                    sb.append("Plan: ").append(rm.getPlan()).append("\n");
                    sb.append("Status: ").append(rm.activeStatus() ? "Active" :
"Inactive").append("\n\n");
                }
            }

            sb.append("Premium Members\n");
            for (GymMember member : membersList) {
                if (member instanceof PremiumMember) {
```

```
                    PremiumMember pm = (PremiumMember) member;
                    sb.append("ID: ").append(pm.getId()).append("\n");
                    sb.append("Name: ").append(pm.getName()).append("\n");
                    sb.append("Status: ").append(pm.activeStatus() ? "Active" :
"Inactive").append("\n\n");
                }
            }
        }

        area.setText(sb.toString());
    }

    private void saveToFile() {
        try {
            FileWriter fileWriter = new FileWriter("members.txt");
            BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);

            // Write header
            bufferedWriter.write(String.format("%-5s %-15s %-15s %-15s %-25s %-10s
%-20s %-15s %-10s %-15s %-10s %-15s %-15s %-15s\n",
                    "ID", "Name", "Location", "Phone", "Email", "Gender", "Membership Start
Date",
                    "Plan", "Price", "Attendance", "Loyalty Points", "Active Status", "Full
Payment", "Paid Amount"));

            // Write member details
            for (GymMember member : membersList) {
                if (member instanceof RegularMember) {
                    RegularMember rm = (RegularMember) member;
```

```
                    bufferedWriter.write(String.format("%-5d %-15s %-15s %-15s %-25s %-10s %-20s %-15s %-10.2f %-10d %-15.2f %-10s %-15s %-15s\n",
                            rm.getId(), rm.getName(), rm.location(), rm.phone(), rm.email(),
                            rm.gender(), rm.membershipStartDate(), rm.getPlan(), rm.getPrice(),
                            rm.attendance(), rm.loyaltyPoints(), rm.activeStatus() ? "Active" : "Inactive",
                            "N/A", "N/A"));
                } else if (member instanceof PremiumMember) {
                    PremiumMember pm = (PremiumMember) member;
                    bufferedWriter.write(String.format("%-5d %-15s %-15s %-15s %-25s %-10s %-20s %-15s %-10s %-10d %-15.2f %-10s %-15s %-15.2f\n",
                            pm.getId(), pm.getName(), pm.location(), pm.phone(), pm.email(),
                            pm.gender(), pm.membershipStartDate(), "Premium", "50000.00",
                            pm.attendance(), pm.loyaltyPoints(), pm.activeStatus() ? "Active" : "Inactive",
                            pm.isFullPayment? "Yes":"No", pm.paidAmount));
                }else{
                    continue;
                }


            }


            bufferedWriter.close();
            JOptionPane.showMessageDialog(frame, "Member details saved to members.txt successfully!");
        } catch (IOException e) {
            JOptionPane.showMessageDialog(frame, "Error saving to file: " + e.getMessage());
        }
    }
```

```
// Method to read members from file and display
private void readFromFile() {
    JFrame readFrame = new JFrame("Members from File");
    readFrame.setSize(800, 600);
    readFrame.setLayout(new BorderLayout());

    JTextArea textArea = new JTextArea();
    textArea.setEditable(false);
    JScrollPane scrollPane = new JScrollPane(textArea);

    try {
        FileReader fileReader = new FileReader("members.txt");
        BufferedReader bufferedReader = new BufferedReader(fileReader);

        String line;
        while ((line = bufferedReader.readLine()) != null) {
            textArea.append(line + "\n");
        }

        bufferedReader.close();
    } catch (IOException e) {
        textArea.setText("Error reading file: " + e.getMessage());
    }

    readFrame.add(scrollPane, BorderLayout.CENTER);

    JButton closeButton = new JButton("Close");
```

```
        closeButton.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e) {
                    readFrame.dispose();
                }
            });

        JPanel buttonPanel = new JPanel();
        buttonPanel.add(closeButton);
        readFrame.add(buttonPanel, BorderLayout.SOUTH);

        readFrame.setVisible(true);
    }

}
```